



# SISTEMA FIWARE EN TIEMPO REAL

VÍCTOR CARRASCO PATO



# POLITÉCNICA

15 DE JULIO DE 2022

UNIVERSIDAD POLITÉCNICA DE MADRID  
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA DE SISTEMAS  
INFORMÁTICOS

## **TRABAJO DE FIN DE GRADO**

Grado en Ingeniería de Computadores

Curso académico 2021-2022

***Autor: Víctor Carrasco Pato***

*Tutores: Borja Bordel Sánchez y Ramón Pablo Alcarria Garrido.*

## Resumen

El Internet de las cosas (IoT), ese término tan escuchado alrededor del mundo, pero realmente, ¿Qué es? Este término describe la red de objetos físicos interconectados de diferentes maneras entre sí, así como sensores, diferentes componentes hardware, software y otras posibles tecnologías. Estos sistemas basados en IoT están implementados en cualquier sitio alrededor del mundo, desde la tienda de la esquina a la que bajamos a comprar el pan hasta los cohetes mandados en las misiones espaciales de la NASA y SpaceX. Dada la extensa población de estos dispositivos no es de extrañar la variedad de los mismos que existe, se puede encontrar desde un sistema con un simple sensor para medir la temperatura de una habitación hasta un sistema que esté leyendo todos los sensores de un coche y actuando en consecuencia de lo que estos sensores detectan.

Los sistemas IoT pueden desarrollarse de muchas formas utilizando un amplio abanico de tecnologías diferentes tales como Big Data, Block Chain, Inteligencia Artificial, Bases de Datos persistentes... Esta última tecnología llama la atención por su capacidad de almacenar los datos tomados por los sensores de forma persistente y a su vez usar los datos en tiempo real. Por otra parte, el tipo de conectividad usado en estos sistemas es muy importante ya que es lo que va a permitir usar los datos de los sensores de forma eficiente y rápida. Se pueden encontrar diferentes tipos de conectividad en estos sistemas como, por ejemplo: Conectividad física guiada por cable, Wifi, LoRa, etc...

Dada toda esta información, se va a desarrollar en este proyecto un sistema basado en IoT con el objetivo de controlar de luz, humedad, temperatura y polución del ambiente permitiendo así controlar las estadísticas del entorno en el que se instale el sistema. Estos datos, además, podrían ser usados por organismos oficiales como medidas ambientales de la zona. La idea es que este proyecto contenga algunas de las tecnologías que se han mencionado en el resumen. Por una parte, los datos percibidos en el sistema han de ser concurrentes y se deben de guardar en una base de datos persistente usando la tecnología FIWARE. Por otra parte, se ha decidido que el tipo de conectividad usada en el proyecto sea LoRa (Long Range), una tecnología de modulación de amplio espectro, lo que le permite tolerar ruido, múltiples caminos de señal y doblegar el efecto Doppler a la vez que consigue conservar un bajo consumo de energía. El único contra que tiene esta tecnología es el reducido ancho de banda que mantiene a diferencia de otras tecnologías inalámbricas como WiFi por ejemplo. Se espera que este proyecto de como resultado un sistema eficiente que pueda operar durante todas las horas del día manteniendo un bajo coste de energía durante su periodo de funcionamiento y promoviendo así el uso de tecnologías de bajo consumo manteniendo un bajo índice de contaminación en este tipo de sistema.

## Summary

The Internet of Things (IoT), that term so heard around the world, but what is it? This term describes the network of physical objects interconnected in different ways with each other, as well as sensors, different hardware components, software and other possible technologies. These IoT-based systems are deployed everywhere around the world, from the corner store where we go down to buy bread to the rockets sent on NASA and SpaceX space missions. Given the large population of these devices, it is not surprising the variety of them that exists, it can be found from a system with a simple sensor to measure the temperature of a room to a system that is reading all the sensors of a car and acting as a result of what these sensors detect.

IoT systems can be developed in many ways using a wide range of different technologies such as Big Data, Block Chain, Artificial Intelligence, Persistent Databases... This latest technology draws attention for its ability to store data taken by sensors in a timely manner and in turn use the data in real time. On the other hand, the type of connectivity used in these systems is very important since it is what will allow the sensor data to be used efficiently and quickly. Different types of connectivity can be found in these systems, such as: Physical connectivity guided by wire, Wifi, LoRa, etc...

Given all this information, an IoT-based system will be developed in this project with the aim of controlling light, temperature, humidity and environmental pollution, thus allowing control of the viability of the area in which the system is installed. These data, in addition, could be used by official organizations as environmental measures of the area. The idea is that this project contains some of the technologies that have been mentioned in the summary. On the one hand, the data received in the system must be concurrent and must be stored in a persistent database using FIWARE technology. On the other hand, it has been decided that the type of connectivity used in the project is LoRa (Long Range), a broad-spectrum modulation technology, which allows it to tolerate noise, multiple signal paths and bend the Doppler effect while managing to preserve a low energy consumption. The only drawback of this technology is the low bandwidth that it maintains, unlike other wireless technologies such as WiFi, for example. It is expected that this project will result in an efficient system that can operate during all hours of the day, maintaining a low energy cost during its period of operation and thus promoting the use of low consumption technologies while maintaining a low rate of pollution in this type of systems.

## Índice de contenidos

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Introducción y Objetivos .....</b>                    | <b>1</b>  |
| 1.1. Introducción .....                                     | 1         |
| 1.2. Objetivos .....                                        | 2         |
| <b>2. Estado del Arte .....</b>                             | <b>4</b>  |
| 2.1. Contexto actual .....                                  | 4         |
| 2.2. Fundamentos del IoT e IoE .....                        | 4         |
| 2.2.1. Modelo de capas del IoT .....                        | 5         |
| 2.2.2. Otras capas de los sistemas IoT .....                | 7         |
| <b>3. Implementación Software .....</b>                     | <b>9</b>  |
| <b>4. Funcionamiento del sistema .....</b>                  | <b>20</b> |
| 4.1. Funcionamiento de la ESP32 .....                       | 20        |
| 4.2. Funcionamiento del <i>Middleware</i> .....             | 22        |
| 4.3. Funcionamiento de Orion Context Broker .....           | 24        |
| 4.4. Funcionamiento de ThingsBoard .....                    | 26        |
| <b>5. Implementación Hardware .....</b>                     | <b>30</b> |
| 5.1. Dispositivos del Sistema .....                         | 30        |
| 5.2. Esquema del sistema .....                              | 35        |
| <b>6. Resultados .....</b>                                  | <b>37</b> |
| 6.1. Resultados del Orion Context Broker .....              | 37        |
| 6.2. Resultados de ThingsBoard .....                        | 38        |
| <b>7. Aspectos éticos, legales y medioambientales .....</b> | <b>41</b> |
| <b>8. Conclusiones .....</b>                                | <b>43</b> |
| 8.1. Desafíos encontrados .....                             | 43        |
| 8.2. Conocimientos adquiridos .....                         | 43        |
| 8.3. Líneas futuras .....                                   | 44        |
| <b>9. Bibliografía .....</b>                                | <b>45</b> |

## Índice de figuras

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Ilustración 1: Capas IoT .....                                             | 6  |
| Ilustración 2: Gráfica Resistencia/Luminancia .....                        | 20 |
| Ilustración 3: Gráfica Resistencia/Contaminante.....                       | 21 |
| Ilustración 4: Esquema del sistema .....                                   | 22 |
| Ilustración 5: Inicialización del <i>middleware</i> .....                  | 23 |
| Ilustración 6: Datos enviados al <i>middleware</i> .....                   | 23 |
| Ilustración 7: Plantilla de datos recibidos por el <i>middleware</i> ..... | 24 |
| Ilustración 8: Funcionamiento Orion Context Broker .....                   | 25 |
| Ilustración 9: Datos recibidos dentro del OCB .....                        | 25 |
| Ilustración 10: Comando de creación de entidad .....                       | 26 |
| Ilustración 11: Añadir dispositivo Thingsboard.....                        | 27 |
| Ilustración 12: Mostrar datos Thingsboard .....                            | 27 |
| Ilustración 13: Configuración de gráfica .....                             | 28 |
| Ilustración 14: Token de entidad.....                                      | 28 |
| Ilustración 15: Módulo ESP32 .....                                         | 30 |
| Ilustración 16: Esquema de pines ESP32 .....                               | 31 |
| Ilustración 17: Sensor DHT11 .....                                         | 32 |
| Ilustración 18: Conexiones sensor DHT11.....                               | 32 |
| Ilustración 19: Fotorresistor .....                                        | 33 |
| Ilustración 20: Sensor MQ-135.....                                         | 33 |
| Ilustración 21: Conexiones sensor MQ-135 .....                             | 34 |
| Ilustración 22: Recreación del sistema físico .....                        | 35 |
| Ilustración 23: Sistema físico real.....                                   | 36 |
| Ilustración 24: Muestra de datos en el monitor serie.....                  | 37 |
| Ilustración 25: Muestra de datos en el <i>middleware</i> .....             | 37 |
| Ilustración 26: Muestra de datos almacenada en el OCB .....                | 38 |
| Ilustración 27: Muestra de datos en el monitor serie.....                  | 39 |
| Ilustración 28: Muestra de datos en Thingsboard .....                      | 39 |

# 1. Introducción y Objetivos

## 1.1. Introducción

Durante los últimos 77 años la evolución de la tecnología ha sido imparable. Esta era comenzaba con la creación del primer ordenador en el año 1945 con el objetivo de estudiar la viabilidad de algunas armas nucleares durante la Segunda Guerra Mundial. Irónicamente la creación de un dispositivo diseñado para eliminar vidas indirectamente, acabo siendo el precedente de la gran era tecnológica que le sucedería años después: el ordenador.

Poco después, en 1958 Geoffrey Dummer, científico de radares, conceptualiza el primer circuito integrado, también conocido como chip. Un circuito integrado es una pequeña estructura hecha de materiales semiconductores, esta contiene diferentes circuitos integrados que interconectan los componentes electrónicos que se encuentran soldados en la placa. La creación de este chip primer chip sería el precedente para los sistemas embebidos que posteriormente serían creados.

Los sistemas embebidos son placas que contienen diferentes circuitos integrados y componentes interconectados, pero a diferencia de los chips estos componentes electrónicos están controlados por un microprocesador capaz de procesar datos y utilizar los componentes empotrados en la placa para desarrollar la función para la cual ha sido designado. Estos sistemas empezaban a usarse en los primeros aparatos electrónicos los electrodomésticos, diferentes sistemas electrónicos de los coches, sistemas de control instalados en las torres de telecomunicaciones, etc.

Unos años más tarde, estos sistemas embebidos evolucionarían evitablemente, dando a luz a nuevos sistemas con placas centrales capaces de controlar diferentes dispositivos conectados a ellos como sensores, *displays* u otros dispositivos electrónicos a través de un software implantado en su microcontrolador. Estos sistemas son capaces de conectarse a la red a través de la conectividad cableada o inalámbrica, incluso de conectarse a otros dispositivos sin necesidad de cables, enviando y recibiendo datos de los diferentes dispositivos electrónicos. Estos sistemas pasarían a llamarse a lo que hoy en día se conoce como Internet de las cosas (IoT).

Dado todo este contexto de la evolución de los sistemas basados en IoT, no es de extrañar la iniciativa mostrada de desarrollar un sistema de este tipo como trabajo de fin de grado. Por una parte, estos sistemas ofrecen una cantidad de alternativas, objetivos y funcionalidades prácticamente ilimitadas por lo que se hace muy viable desarrollar un sistema que tenga un objetivo concreto sin demasiado costo económico. Por otra parte, el hecho de haber estudiado durante los cuatro últimos años un grado en Ingeniería de Computadores el cual se centra en la enseñanza de los conocimientos sobre el hardware genera gran entusiasmo a la hora de realizar este proyecto, dando este una oportunidad de demostrar los conocimientos adquiridos durante todos estos cuatro años.

El sistema que se va a desarrollar en este proyecto está conectado a la red a través de la tecnología de conectividad LoRa, la cual nos permitirá enviar los datos recogidos por los sensores del sistema a un servidor alojado en un equipo que actuará como host. Al utilizar la tecnología LoRa se evitarán numerosos obstáculos que suelen resultar en errores de conexión

y por lo tanto no se da un servicio de calidad por parte del sistema, entre estos obstáculos cabe destacar que la tecnología LoRa es capaz incluso de evitar algunos fenómenos físicos como es la presencia de ruido en el ambiente o el efecto doppler. Los datos recogidos por el servidor serán registrados en una base de datos persistente contextual gracias a la tecnología Quantumleap.

Quantumleap es una tecnología creada por la empresa FIWARE, empresa dedicada crear soluciones tecnológicas con la peculiar característica de dar todas sus implementaciones en código abierto. Esta tecnología no solo es capaz de almacenar y consultar datos, sino también de recuperar datos espaciotemporales, lo que quiere decir que los datos recuperados de la base de datos tendrán un índice que delatará de dónde proviene ese dato y cuándo fue tomado.

Teniendo en cuenta esta capacidad tecnológica que ofrece Quantumleap un sistema con unos sensores orientados a la medición de los factores ambientales en un entorno al aire libre sería el objeto de medición ideal para esta tecnología. La capacidad de Quantumleap de proporcionar estos datos con un índice espaciotemporal hace que la medición medioambiental en un entorno al aire libre sea perfecta como objetivo de medición del sistema, pudiendo ofrecer información a tiempo real y contextualizada en el tiempo y el espacio, y pudiendo comparar así la calidad medioambiental sobre diferentes puntos de España.

## 1.2. Objetivos

El objetivo de este proyecto es desarrollar un sistema basado en IoT haciendo que todos los sensores pertenecientes al sistema envíen los datos a la red a través de la conectividad inalámbrica Lora y sean almacenados en una base de datos persistente y contextual mediante la tecnología ofrecida por FIWARE Quantumleap.

Como objetivo de medición se ha escogido a los factores medioambientales como son la luminancia, la calidad del aire, la temperatura y la humedad. Estas mediciones darán una idea de la calidad medio ambiental en diferentes puntos de la península española pudiendo ofrecer tablas y gráficas comparativas proporcionando datos a lo largo del tiempo y pudiéndolos diferenciar con otros datos recogidos en los mismos periodos de tiempo del día, pero en diferentes geolocalizaciones.

A pesar de que numerosas estaciones ambientales distribuidas alrededor del mundo tienen esta capacidad, se quiere demostrar que cualquier tipo de sensores conectados a una placa compatible pueden ser parte de un sistema funcionando con esta innovadora tecnología de Quantumleap.

Utilizando esta tecnología se pretende también promover los proyectos de la mano de FIWARE dando la oportunidad a los desarrolladores de poder usar nuevas tecnologías con código abierto lo que abriría una gran cantidad de posibilidades a la hora de desarrollar este tipo de sistemas en comparación con los sistemas desarrollados con herramientas de código cerrado que limitan bastante la capacidad de obtener nuevas funcionalidades por parte de los sistemas.

A continuación, se van a enumerar los hitos a completar en el proyecto, los hitos futuros del proyecto se comentarán junto a la conclusión:

- Desarrollar entorno software capaz de conectar la placa esp32 a la red y enviar los datos de los sensores al servidor establecido.
- Crear un servidor en la máquina local que sea capaz de almacenar los datos recogidos por el sistema y guardarlos en una base de datos.
- Habilitar la herramienta Orion Context Broker para poder registrar los cambios en el sistema y las mediciones durante el funcionamiento del mismo.
- Hacer que el sistema sea portable, es decir, que el sistema pueda funcionar en cualquier punto sin necesidad de una tediosa instalación, simplemente conectando el sistema a una fuente de alimentación portátil y conectándose a la red a través de un punto de acceso móvil.
- Comparar los resultados obtenidos por el sistema desde diferentes geolocalizaciones de la península ibérica y en las mismas franjas horarias.
- Estudiar la huella de carbono que dejaría la construcción y funcionamiento del sistema.
- Estudiar la implementación de futuras funcionalidades para el sistema.

## 2. Estado del Arte

### 2.1. Contexto actual

El internet de las cosas es un nuevo ejemplar dentro del mundo del IT (*Information Technology*). La frase “internet de las cosas” está formada por dos palabras “internet y “cosas”. Por una parte, “Internet” es un sistema global masivo e interconectado alrededor del globo el cual está conformado por millones de diferentes redes públicas y privadas de todos los tipos de alcance desde nivel local hasta nivel global. Hoy en día más de 100 países alrededor del mundo poseen la tecnología para conectarse a internet creando así una red aún más grande llena de conocimientos y tecnología. Según Internet World Statistics, para diciembre de 2011 había aproximadamente una cantidad de 2.700.000.000 usuarios de Internet alrededor del globo. Por otra parte, con la palabra “cosas” se puede referir a cualquier objeto tangible de nuestra vida cotidiana, no solo nos referimos a dispositivos electrónicos que usamos en nuestro día a día, sino también aquellos objetos animados e inanimados, como puede ser un animal, una persona, un sofá o una mesa. Dado este punto referencial, se puede afirmar con seguridad que los sistemas IoT se van a centrar en dar soluciones con diferentes implementaciones a cualquier tipo de objeto de cualquier tipo de contexto diferente en nuestro planeta.

A lo largo de los años se ha intentado dar una definición a esta frase, pero diferentes expertos y empresas no son capaces de coincidir en una definición concreta del internet de las cosas. La única conclusión a la que llegaron a estar en común fue que la idea de internet en una primera instancia producía datos provenientes de personas, mientras que el internet de las cosas se trata de una versión que da ofrece datos provenientes de “cosas”. Esta definición fue la siguiente: “Los sistemas IoT se tratan de una red abierta y llena de objetos inteligentes capaz de gestionarse a sí misma, mandar y recibir información, y actuar en consecuencia de cualquier situación que pueda surgir.”

El internet de las cosas ha sido capaz de empezar una revolución de la tecnología que pone expectativas en el futuro de la ciencia y los nuevos descubrimientos en este ámbito, trayendo consigo incluso nuevas innovaciones en diferentes campos tecnológicos de las industrias existentes. Uno de los primeros dispositivos conectados a internet fue una máquina perteneciente a la compañía “Coca-Cola”, esta máquina era una expendedora de bebidas que se instaló en la Universidad de Carnegie Mellon, Pittsburgh, Pensilvania. Los desarrolladores que trabajaron con el sistema implantado en esta máquina codificaron su funcionamiento de tal forma que cuando una fila de productos se agotaba el sistema enviaba un aviso al servidor establecido, de esta forma medían el tiempo que tardaba en volver a vaciarse determinando así cual sería el momento perfecto para poder coger una bebida bastante fría.

### 2.2. Fundamentos del IoT e IoE

Como ya se ha explicado antes el IoT se define como una red de objetos físicos interconectados entre sí con un propósito final común. Estos objetos físicos en la actualidad pueden llegar a ser cualquier tipo y tamaño de objeto, como pueden ser teléfonos, naves industriales, animales, personas e incluso edificios.

Habiendo ya indagando profundamente en lo que es la tecnología IoT, se va a pasar a explicar un término muy nuevo, y este es IoE, o lo que es lo mismo: Internet del todo (*Internet of*

*Everything*). Esta novedosa tecnología no solo provee enlaces entre las cosas, sino que también lo hace entre datos, personas y procesos. Esta tecnología destaca porque los organismos vivos pueden ser insertados en un sistema lógico de intercambio de datos haciendo que se envíen y reciban datos a los correspondientes sistemas electrónicos responsables de medir de la actividad y comportamiento de los organismos vivos. Un gran ejemplo de esto es el proyecto *The Cow Tracking Project* (Proyecto de seguimiento de la vaca). Este proyecto convoca una gran expectación debido a lo innovador que es en el campo de la tecnología IoT/loE debido a que usan las vacas como producto de medida, registrando continuamente datos para verificar la geolocalización de las vacas y buscar diferentes patrones de comportamientos entre los distintos sujetos. Estos datos recogidos por los desarrolladores del proyecto son enviados por unos dispositivos electrónicos (*wereables*) capaces de enviar distintos datos del sujeto animal en tiempo real. A su vez estos sujetos están conectados a internet a través de unas etiquetas de posicionamiento vía radio que permiten realizar un seguimiento en tiempo real del animal. Sin embargo, en los sistemas loE los datos no son los únicos que juegan un papel importante, sino que también los procesos por los cuales se envían esos datos juegan un papel crucial, es decir que, si los procesos utilizados en el sistema son los correctos, la información compartida adquiere mucho mas valor ya que la información es compartida en el momento y al receptor adecuado.

Por últimos vamos a cerrar la introducción a las loE nombrando los 4 pilares en los que se basa:

- **Las personas:** Conectado gente de una forma más valiosa y relevante.
- **Los datos:** Convirtiendo los datos en inteligencia para tomar mejores decisiones.
- **Los procesos:** Enviando la información al destinatario correcto en el momento oportuno.
- **Las cosas:** Dispositivos físicos y objetos interconectados entre ellos y a su vez conectados a internet.

### 2.2.1. Modelo de capas del IoT

En la ilustración 1 se muestra una abstracción de la tecnología IoT en forma de modelo de capas. De esta forma será más fácil explicar las diferentes agrupaciones de funcionalidades que tiene esta tecnología, si bien es cierto que no hay un modelo estandarizado de capas referente a la abstracción de la tecnología I.T ya que los diferentes proveedores y desarrolladores tienen sus propios estándares sobre la estructura abstracta de esta tecnología, en esta memoria se va a intentar generalizar las diferentes ideas que se tienen sobre estas capas.

## THREE-LAYER ARCHITECTURE

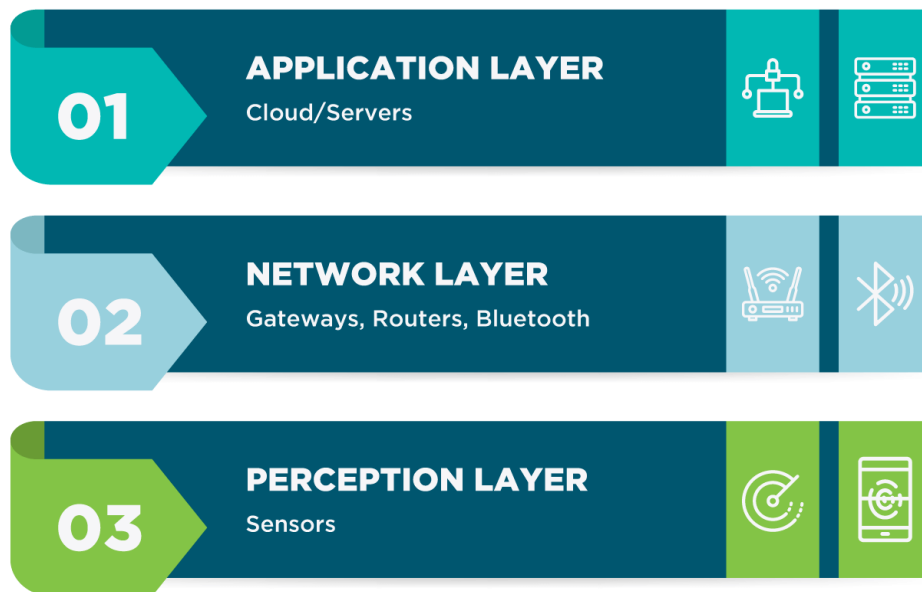


ILUSTRACIÓN 1: CAPAS IoT

Como se puede observar las capas de aplicación, comunicación y percepción son las tres capas en las que coinciden la mayoría de los desarrolladores y proveedores de este tipo de sistemas. Ahora se va a pasar a explicar en los que consiste cada una de las capas mostrada en la ilustración 1:

- **Capa de Percepción:** O también llamada capa sensorial, esta capa recoge los sensores físicos que conforman un sistema IoT. El objetivo de estos dispositivos es obtener información del entorno y luego procesarla y convertirla en una información legible para el ser humano. Esta capa provee diferentes funcionalidades:
  - Funciones naturales de los sensores como son la medición y captura de datos del entorno físico y llevarlos al mundo de los datos.
  - Funciones físicas o animadas por parte de los actuadores, los cuales reciben una información, orden o proceso y actúan en consecuencia realizando algún tipo de movimiento para el cual estén programados.
  - Funciones Edge que permiten el procesamiento de cierta información compleja
  - Funciones Fog que conllevan a procesamientos y almacenamientos que requieren un elevado rendimiento y carga de trabajo.
- **Capa de Comunicación:** Esta capa se ocupa de la transferencia de los datos obtenidos en la capa de percepción y los envía a la capa de aplicación. A esta capa se le imponen las siguientes funcionalidades:

- Función de abstracción respecto a la conectividad, encapsulando los metadatos de las tecnologías utilizadas. Se llegan a usar APIs para proponer diferentes servicios de conectividad.
  - Funciones de comunicaciones que exigen el uso de unos conectores mandatorios para todos los protocolos utilizados en estos sistemas.
  - Funciones de ciberseguridad empleando diferentes protocolos de seguridad que afecten a la propiedad y condición de los datos usados.
- **Capa de Aplicación:** O también conocida como capa de soporte, esta capa se dedica a la gestión de servicios y datos recibidos por las otras capas. Esta capa confiere a los desarrolladores la capacidad de programar la completa funcionalidad del sistema con una serie de líneas de código. Al igual que las capas anteriormente mencionadas, esta capa posee las siguientes funcionalidades:
  - Funcionalidad de gestión de componentes electrónicos o dispositivos, siendo capaz de almacenar, administrar y monitorear la información de los mismos.
  - Gestión del acceso al medio de la información generada por los dispositivos del sistema.
  - Seguimiento en tiempo real de los eventos que son provocados en el sistema.

## 2.2.2. Otras capas de los sistemas IoT

Como bien se ha comentado en el apartado anterior, las tres capas mencionadas son aquellas en las que coinciden la mayoría de los proveedores y desarrolladores de sistemas IoT, pero hay muchas otras capas que dichas entidades comentan y que tienen la relevancia suficiente en el entorno de estos sistemas como para explicarlas en este documento:

- **Capa de negocio:** Esta capa es utilizada para proveer al sistema de los servicios que están siendo solicitados en el momento por parte del usuario. Esta capa está dotada con diferentes funciones que involucran prestar servicios e interactuar con la capa de aplicación anteriormente mencionada. La capa de negocio ofrece la siguiente funcionalidad:
  - Capacidad de diseño y desarrollo a través de APIs (*Application programming interface*) para desarrollar aplicaciones internas necesarias para el propio sistema.
- **Capa de Procesamiento:** Esta capa controla y administra los diferentes niveles del sistema IoT transportando los datos a lo largo del sistema. En esta capa existen dos escenarios:
  - Acumulación de datos: En este escenario se reúne todo lo referido con la totalidad de las tramas de datos enviadas por los diferentes sensores del sistema a través de la red IoT. Tal es la cantidad de datos que se envían durante el funcionamiento del sistema, además de las diferentes formas y velocidades en el que la información es recibida, que separar las grandes tramas de la información esencial es una prioridad en este tipo de sistemas.

- o **Abstracción de datos:** Una vez el escenario de la acumulación de datos ha terminado, se procede a abstraer los datos, es decir, a coger los datos importantes de las grandes tramas que cruzan de un lado al otro del sistema. Los pasos que se siguen para llevar a cabo esta abstracción de datos son los siguientes:
    - Recoger los datos de todos los sistemas ya sean o no IoT.
    - Usar la virtualización de datos para hacer que la información sea accesible desde cualquier sitio.
    - Administrar los datos no procesados de múltiples maneras.
- **Capa de seguridad:** Esta capa, como bien indica el nombre, contiene la seguridad de toda la arquitectura IoT, desde el propio código y funcionamiento, hasta los sensores del sistema. Esta capa se involucra en los distintos niveles de seguridad que pueda tener el sistema:
  - o **Seguridad de los dispositivos:** Es el nivel primario de seguridad de esta capa ya que lo primero que hay que proteger es el propio sistema IoT. Estas son las medidas generalmente tomadas por las empresas y los profesionales:
    - Proceso de inicio seguro para evitar que cualquier código malicioso se esté ejecutando durante el inicio del sistema.
    - Usar chips TPM (*Trusted Platform Module*) junto a claves criptográficas.
    - Existencia de una capa física extra para evitar el acceso directo a los dispositivos.
    - Actualizaciones de seguridad regulares.
  - o **Seguridad de la nube:** Ahora que están tan de moda los sistemas de almacenado en la nube, se hace imposible pensar que un sistema IoT pueda carecer de un almacenamiento de datos en la nube. Por lo que es imprescindible que tengamos el sistema de almacenamiento en la nube bien salvaguardado con herramientas y mecanismos como es la autenticación en dos factores y las encriptaciones para evitar cualquier brecha de seguridad.
  - o **Seguridad de conexión:** Cuando los datos del sistema IoT se transfieren a lo largo de la red, estos deben de estar encriptados de extremo a extremo. En este nivel de seguridad, los protocolos DDS, AMQP y MQTT están integrados como protocolos de seguridad de la red.

### 3. Implementación Software

En este apartado se va a explicar el desarrollo software del proyecto que implica la programación de los diferentes dispositivos, el funcionamiento del *middleware* y la puesta a punto de Orion Context Broker.

Se procede a explicar el código implementado para el correcto funcionamiento del sistema. El código utilizado en el sistema se rige por dos lenguajes de programación: C y JavaScript.

Se empezará con la explicación del código referente a la ESP32:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <ThingsBoard.h>
#include <DHT.h>
```

En las primeras líneas del código se están incluyendo las librerías que se van a usar en el programa:

- **WiFi.h:** Librería usada para conectar la ESP32 a la red. Esta librería trata las prestaciones wifi de la placa permitiéndole a esta conectarse a internet y usar otro tipo de prestaciones como es el crear un servidor instanciado, un cliente instanciado e incluso enviar y recibir paquetes UDP a través del WiFi.
- **HTTPClient.h:** Esta biblioteca provee al desarrollador la posibilidad de hacer comandos HTTP a cualquier servidor de la red, estos comandos son los siguientes: GET, POST y PUT. Está librería se usará para poder subir los datos de los sensores captados por el sistema al *middleware* establecido.
- **ArduinoJson.h:** Esta librería contiene diferentes herramientas y funciones para tratar a los objetos y variables del código como objetos JSON (JavaScript Object Notation). En el proyecto se utilizará esta librería para serializar los datos mandados por los sensores y que estos sean captados por el *Middleware*.
- **ThingsBoard.h:** Esta librería se utiliza principalmente para conectar el sistema a la plataforma IoT ThingsBoard. La librería permite mandar los diferentes datos recogidos por los sensores a la plataforma con tan solo proveer la URL de la web y la API Key del dispositivo interno al que se vayan a mandar los datos.
- **DHT.h:** La siguiente biblioteca se relaciona con los sensores DHT, es decir sensores capaces de detectar la humedad y temperatura del entorno. Esta biblioteca proporciona herramientas para recoger los datos que manda el sensor al sistema, ya que estos datos se encuentran comprimidos y la librería contiene funciones capaces de descomprimir estos datos y poder mostrarlos por separado.

```
StaticJsonDocument<500> doc;
WiFiClient client;

const char* ssid = "MiFibra-1FC9-2.4G";
const char* password = "Pv2fvG3R";

const char* serverName = "http://192.168.1.92:3000/ip";

unsigned long lastTime = 0;
unsigned long timerDelay = 30000;
```

A continuación, se encuentran las variables globales que se utilizarán a lo largo del código:

- StaticJsonDocument<500> doc: Variable declarada como Documento Json con una capacidad de 500 caracteres. Esta variable se usará para almacenar los datos recogidos por los sensores.
- const char\* ssid & password: En esta variable se almacena un string con los datos del punto de acceso WiFi al que se quiera conectar la ESP32 y la correspondiente contraseña del punto de acceso.
- const char\* serverName: En esta variable se almacena el punto final de conexión del sistema, en el caso de este proyecto se ha especificado la dirección 192.168.1.92 con puerto 3000 y camino /ip, apuntando así a la dirección del *middleware* del sistema.
- unsigned long lastTime & timerDelay: Por último, estas variables globales almacenan un entero cada una (0 y 30000 correspondientemente). Estas variables controlarán el tiempo que tarda el sistema en volver a enviar una trama HTTP al *middleware*.
- WiFiClient client: Se declara el la variable "client" como objeto tipo "WiFiClient".

```
const char* TOKEN = "VNbTk60funcwJz2REj11";
const char* THINGSBOARD_SERVER = "demo.thingsboard.io";
ThingsBoard tb(client);
bool subscribed = false;
```

A continuación, se declaran las variables necesarias para interactuar con ThingsBoard:

- const char\* TOKEN: Variable que almacena los caracteres componentes de la API Key referente al dispositivo de ThingsBoard, con esta clave al enviar los datos a ThingsBoard, estos acabarán almacenados en el dispositivo al que se le liga esta clave.
- const char\* THINGSBOARD\_SERVER: En la siguiente variable se guarda la URL correspondiente a la plataforma concreta de ThingsBoard a la que se va a conectar el sistema.
- ThingsBoard tb(client): Se declara la variable "tb" como objeto tipo "ThingsBoard" con la variable "client" formando parte del constructor
- bool subscribed: Variable de tipo booleano que se utiliza para verificar si el sistema se mantiene conectado al servidor de la plataforma ThingsBoard.

```
#define DHTPIN 32
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

Ahora se declaran las variables correspondientes al funcionamiento del sistema con el sensor DHT11:

- #define DHTPIN 32: Esta variable almacena el valor referente al pin de entrada/salida de la ESP32 número 32 (GPIO32).
- #define DHTTYPE DHT11: Aquí se define el tipo de sensor que se va a utilizar en el sistema ya que esta biblioteca contiene funciones para distintos sensores de la familia DHT, en el caso de este proyecto se selecciona que el sensor deseado es el DHT11.
- DHT dht(DHTPIN, DHTTYPE): Se declara el objeto "dht" con las variables "DHTPIN" y "DHTTYPE" como parte del constructor.

```
int valorLDR = 0;
int pinLDR = 34;
```

A continuación, se declaran las variables usadas para el funcionamiento del fotoresistor:

- int valorLDR: Variable que se usará para almacenar el valor de "pinLDR", solo que este valor ya estará transformado a valor digital.
- int pinLDR: En este pin se almacena el número del pin de entrada/salida de la ESP32, en este caso se apunta al "GPIO34".

```
#define GASPIN 33
int valorGAS = 0;
```

Estas variables son las últimas referentes a los sensores que quedan por declarar, en este caso, las siguientes variables son referentes al sensor de calidad del aire:

- #define GASPIN 33: Se define la variable donde se almacena el valor referente al número de pin de entrada/salida de la ESP32, en este caso se apunta al "GPIO33".
- int valorGAS: Como en la variable "valorLDR", esta variable también se utiliza para almacenar el valor digital que proviene del sensor correspondiente.

Una vez declaradas las variables globales, se encuentra el comienzo de la función "void setup()". Esta función contiene las líneas que se ejecutan una sola vez durante la ejecución del código, dichas líneas de código son las siguientes:

```
Serial.begin(115200);  
dht.begin();
```

Se empieza especificando por qué canal del monitor serie de Arduino se va a poder visualizar la información que se va mostrando durante la ejecución del código. Con el número “115200” se especifica en qué frecuencia de baudios se está enviando la información. En este caso se ha elegido esa frecuencia de baudios concreta ya que el fabricante del módulo ESP32 especifica que es el canal más adecuado para enviar la información con esta placa.

Justo después se inicializa la funcionalidad referente al sensor DHT11. Es necesario hacerlo en dentro de la función “void” ya que solo se necesita inicializar esta funcionalidad una sola vez. Esta inicialización es indispensable para favorecer la correcta detección de datos por parte del sensor de temperatura y humedad.

```
WiFi.begin(ssid, password);  
Serial.println("Connecting");  
while(WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}
```

En las siguientes líneas, con la función “WiFi.begin()”, se empieza a establecer la conexión con el punto de acceso wifi cuyos datos de acceso se han almacenado previamente en las variables “ssid” y “password”.

Justo después, se abre un bucle while con la siguiente condición: “WiFi.status() != WL\_CONNECTED”. Esta condición implica que mientras el estado de la conexión Wifi no sea “WL\_CONNECTED”, no se saldrá del bucle y se imprimirá por el monitor serie el siguiente carácter “.” cada 0,5 segundos hasta que la conexión con el punto wifi sea establecida.

```
Serial.print("Connected to WiFi network with IP Address: ");  
Serial.println(WiFi.localIP());
```

Una vez la ESP32 ha sido capaz de conectarse al punto Wifi, se va a sacar por el monitor serie la información de que el sistema se ha conectado a la red junto a la dirección IP que se le ha otorgado. Con estas líneas se concluye la función “setup ()”.

A continuación, se abre la función “void loop() “. Esta función contiene las líneas de código que se van a ejecutar de forma repetida e ininterrumpida durante el funcionamiento del sistema. Dichas líneas de código son las siguientes:

```
if ((millis() - lastTime) > timerDelay)
```

Se empieza por un condicional “If” al cual se accede siempre que hayan pasado más de 30 segundos (cantidad almacenada en la variable “timerDelay”) desde la última vez que se almacenó la variable “lastTime”, o lo que es lo mismo, desde la última vez que se enviaron datos al *middleware*.

```
if(WiFi.status()== WL_CONNECTED){  
    HTTPClient http;
```

Una vez se ha verificado que efectivamente han pasado más de 30 segundos del último bucle, se entra en el condicional “If” y se puede encontrar otro condicional “If” justo después. Este último está comprobando si el sistema está conectado al Wifi. Si el sistema permanece conectado al punto de red, entonces se procederá a declarar el objeto http (como objeto “HTTPClient”).

```
float h = dht.readHumidity();  
float t = dht.readTemperature();  
Serial.println("Temperatura: "); Serial.println(t);  
Serial.println("Humedad: "); Serial.println(h);
```

En las líneas de código mostradas arriba, se están declarando las variables que almacenarán los datos recogidos por el sensor de humedad. El objeto “dht” posee las funciones “readHumidity” y “readTemperature” las cuales devuelven el valor de la humedad y la temperatura correspondientemente. Estos dos últimos valores se almacenan en las variables “h” y “t”. Justo después estas variables se muestran junto a un pequeño fragmento de texto referencial a través del monitor serie.

```
valorLDR = analogRead(pinLDR);  
Serial.println("Intensidad de Luz: "); Serial.println(valorLDR);
```

Más adelante están las líneas de código referentes al funcionamiento del fotoresistor. Primero se almacena en la variable “valorLDR” el valor digital proveniente del pin GPIO34 ya que el valor original es de tipo analógico. Una vez almacenado este valor se procede a enseñarlo por pantalla a través del monitor serie junto a un pequeño texto referencial.

```
valorGAS = analogRead(GASPIN);  
Serial.println("Contaminación: "); Serial.println(valorGAS);
```

Como en las dos líneas de código anteriores, se procede a hacer la misma función solo que con las variables referentes al sensor de calidad del aire. Primero se transforma el contenido analógico proveniente del pin GPIO33 en contenido digital. Seguidamente se procede a

enseñar los valores detectados por el sensor de calidad del aire a través del monitor serie junto a un pequeño texto referencial.

```
if (!tb.connected()) {
    subscribed = false;

    // Connect to the ThingsBoard
    Serial.print("Connecting to: ");
    Serial.print(THINGSBOARD_SERVER);
    Serial.print(" with token ");
    Serial.println(TOKEN);
    if (!tb.connect(THINGSBOARD_SERVER, TOKEN)) {
        Serial.println("Failed to connect");
        return;
    }
}
```

A continuación, se abre un condicional en el que se entra si el sistema no se encuentra conectado a ThingsBoard. Si ese es el caso, se procede a intentar conectar el sistema con la plataforma IoT y a continuación, se saca por pantalla tanto la URL del sitio web como la API Key con la que se quiere conectar el sistema a la plataforma.

```
tb.sendTelemetryFloat("temperature", t);

tb.sendTelemetryFloat("humidity", h);
tb.sendTelemetryInt("Luminosidad", valorLDR);
tb.sendTelemetryInt("Contaminación", valorGAS);

tb.loop();
```

En estas líneas de código lo que se hace es usar la función del objeto "tb" llamada "sendTelemetry" especificando el tipo de datos que se esperan. Dentro de esta función se especifica el nombre con el que se quiere guardar al valor que se envía, y al lado se especifica la variable enviada a ThingsBoard.

```
http.begin(client, serverName);
```

A continuación, se establecerá la conexión con la dirección guardada en la variable "serverName", que en este caso es la siguiente: <http://192.168.1.92:3000/ip>.

```
doc["deviceName"] = "Room5";
doc["Calidad del Aire"] = valorGAS;
doc["Luminosidad"] = valorLDR;
doc["Temperatura"] = t;
doc["Humedad"] = h;
```

```
String stg;  
serializeJson(doc,stg);
```

Una vez establecida la conexión con la dirección deseada, se procede a almacenar los datos de los sensores en la variable “doc” (variable tipo Document Json). Una vez almacenados los datos en dicha variable, se procede a serializarla utilizando la función “serializeJson()”. Al serializar esta información lo que se está haciendo es convertir la estructura de datos guardada dentro de la variable “doc” en información que puede ser almacenada, transmitida y reconstruida más tarde. La serialización de esta información se guarda en la variable “stg”.

```
http.addHeader("Content-Type", "application/json");  
int httpResponseCode = http.POST(stg);  
  
Serial.print("HTTP Response code: ");  
Serial.println(httpResponseCode);  
  
http.end();
```

Aquí se presenta el cuerpo principal del código, en el que se implementa el envío de la petición http por la cual se sube la información al *middleware*. En primer lugar, se establecen las cabeceras necesarias que requiere el servidor *middleware* para poder ser alcanzado, en este caso se especifica que el tipo de contenido de cabecera será: “application/json”.

A continuación, se procede a enviar el contenido con las mediciones de los sensores del sistema almacenados en la variable “stg” con la ayuda de la función http.POST (función perteneciente a la librería HTTPClient.h). Esta función creará y enviará una petición http con intención de publicar los datos contenidos en “stg” en el servidor *middleware*.

La variable “httpResponseCode” guardará la respuesta que envía de vuelta el servidor al sistema. Si “httpResponseCode” se devuelve con un valor igual a “200” entonces significará que los datos enviados han sido publicados con éxito.

Por último, con la función “http.end()” lo que se hace es liberar los recursos http y finalizar así la conexión http con el servidor.

```
else {  
    Serial.println("WiFi Disconnected");  
}  
lastTime = millis();  
}
```

Para terminar, se tiene un “else” correspondiente a la comprobación de estado de conexión del sistema que se ha visto anteriormente, por la cual en caso de que el sistema esté desconectado del punto Wifi, este avisará por el monitor serie del estado de desconexión en el que se encuentra. Como última línea de código perteneciente a la función “loop”, se tiene a la variable “lastTime” almacenado el momento en el que se ejecutó el último bucle.

Todo este código explicado anteriormente se muestra de la siguiente manera:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <ThingsBoard.h>
#include <DHT.h>

StaticJsonDocument<500> doc;

const char* ssid = "MiFibra-1FC9-2.4G";
const char* password = "Pv2fvG3R";

//Your Domain name with URL path or IP address with path
const char* serverName = "http://192.168.1.92:3000/ip";

// the following variables are unsigned longs because the time, measured
in
// milliseconds, will quickly become a bigger number than can be stored
in an int.
unsigned long lastTime = 0;
// Timer set to 10 minutes (600000)
//unsigned long timerDelay = 600000;
// Set timer to 5 seconds (5000)
unsigned long timerDelay = 5000;

/////////////////////////////////THINGSBOARD/////////////////////////////////

const char* TOKEN = "VNbTk60funcwJz2REj11";
const char* THINGSBOARD_SERVER = "demo.thingsboard.io";
bool subscribed = false;

/////////////////////////////////

/////////////////////////////////DHT/////////////////////////////////
// Definimos el pin digital donde se conecta el sensor
#define DHTPIN 32
// Dependiendo del tipo de sensor
#define DHTTYPE DHT11
// Inicializamos el sensor DHT11
DHT dht(DHTPIN, DHTTYPE);
/////////////////////////////////

/////////////////////////////////PhotoSensor/////////////////////////////////
/////
// Definimos el pin digital donde se conecta el sensor
//Aquí almacenamos los datos recogidos del LDR:
int valorLDR = 0;
```

```

//Y que pin analógico conectarmos el LDR
int pinLDR = 34;
/////////////////////////////////////////////////////////////////

/////////////////////////////////////////////////////////////////
// Definimos el pin digital donde se conecta el sensor
#define GASPIN 33
int valorGAS = 0;
/////////////////////////////////////////////////////////////////

WiFiClient client;
ThingsBoard tb(client);

void setup() {

    dht.begin(); //DHT SETUP

    Serial.begin(115200);

    WiFi.begin(ssid, password);
    Serial.println("Connecting");
    while(WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.print("Connected to WiFi network with IP Address: ");
    Serial.println(WiFi.localIP());
}

void loop() {
    //Send an HTTP POST request every 10 minutes
    if ((millis() - lastTime) > timerDelay) {
        //Check WiFi connection status
        if(WiFi.status()== WL_CONNECTED){

            //////////////////////////////////DHT////////////////////////////////////
            //
            // Definimos el pin digital donde se conecta el sensor
            float h = dht.readHumidity();
            float t = dht.readTemperature();
            Serial.println("Temperatura: "); Serial.println(t);
            Serial.println("Humedad: "); Serial.println(h);
            //////////////////////////////////
            //

            //////////////////////////////////PhotoSensor////////////////////////////////////
            //

```

```

// Definimos el pin digital donde se conecta el sensor
valorLDR = analogRead(pinLDR);
Serial.println("Intensidad de Luz: "); Serial.println(valorLDR);
////////////////////////////////////
//

//////////////////////////////////GASsensor//////////////////////////////////
/////////
valorGAS = analogRead(GASPIN);
Serial.println("Contaminación: "); Serial.println(valorGAS);
////////////////////////////////////
//

if (!tb.connected()) {
    subscribed = false;

    // Connect to the ThingsBoard
    Serial.print("Connecting to: ");
    Serial.print(THINGSBOARD_SERVER);
    Serial.print(" with token ");
    Serial.println(TOKEN);
    if (!tb.connect(THINGSBOARD_SERVER, TOKEN)) {
        Serial.println("Failed to connect");
        return;
    }
}

    tb.sendTelemetryFloat("temperature", t);
    tb.sendTelemetryFloat("humidity", h);
    tb.sendTelemetryInt("Luminosidad", valorLDR);
    tb.sendTelemetryInt("Contaminación", valorGAS);

    tb.loop();

    HTTPClient http;

    // Your Domain name with URL path or IP address with path
    http.begin(client, serverName);

    String stg;

    doc["deviceName"] = "Room5";
    doc["eCO2"] = random(40);
    doc["TVOC"] = random(40);
    doc["temperatura"] = random(40);
    doc["humedad"] = random(40);
    // doc["distancia"] = random(40);

```

```
Serial.println("WiFi Disconnected");

serializeJson(doc,stg);


//HTTP REQUEST
http.addHeader("Content-Type", "application/json");

int httpResponseCode = http.POST(stg);

Serial.print("HTTP Response code: ");
Serial.println(httpResponseCode);

// Free resources
http.end();

}
else {
  Serial.println("WiFi Disconnected");
}
lastTime = millis();
}
}
```

## 4. Funcionamiento del sistema

En este apartado se va a explicar el funcionamiento del sistema desde la recogida de datos hasta el almacenado y muestra de los mismos.

### 4.1. Funcionamiento de la ESP32

En primer lugar, se va a empezar a explicar la parte referente al sistema físico y el módulo ESP32.

El objetivo de esta parte del sistema es recoger los datos obtenidos por parte de los sensores, transmitirlos a lo largos del sistema, y que finalmente la ESP32 finalmente envíe estos datos tanto a ThingsBoard como al *Middleware*. Estos sensores recogen datos como la temperatura, la humedad, la intensidad de la luz ambiente y la contaminación en el aire. El funcionamiento de estos sensores será explicado a continuación.

Primero se va a comenzar explicando el funcionamiento del sensor de temperatura y humedad. Este sensor es capaz de detectar ambas mediciones al mismo tiempo gracias a los dos sensores integrados que contiene el módulo. Estos datos se envían de forma comprimida a la ESP32 a través de una entrada digital. Una vez la información obtenida por este sensor llega a la ESP32, el código desarrollado junto con la biblioteca DHT.h permite descomprimir la información del sensor y poder separar así los datos obtenidos de la humedad y la temperatura.

El sistema también cuenta con un fotoresistor, que actúa como un sensor de intensidad lumínica. Este fotoresistor tiene una resistencia interna que obstaculiza el paso de la electricidad a través de ella, pero cuando la luz golpea al sensor, la resistencia al paso de la electricidad se reduce, por lo que deja pasar más electricidad a través de él. Esta señal llega a un pin de entrada salida en forma analógica a la ESP32 y el código funcional instalado dentro de la placa se encarga de convertir esta señal en un valor digital y poder así operar con el dato.

En la ilustración 1 se muestra una gráfica sacada del *datasheet* del foto resistor de como este reacciona a la luz: En el eje Y se encuentra la resistencia ofrecida, y en el eje X la escala de luminancia del 1 al 100:

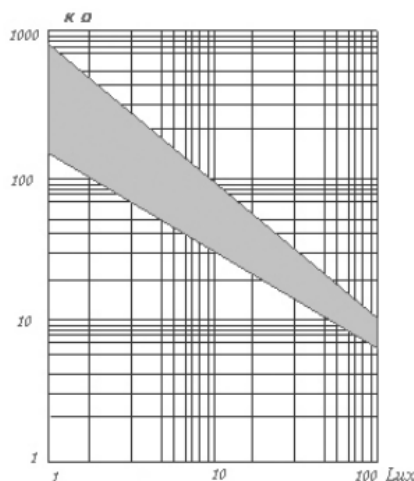


ILUSTRACIÓN 2: GRÁFICA RESISTENCIA/LUMINANCIA

El último sensor del sistema es el encargado de medir la contaminación en el aire. Este dispositivo se llama MQ-135 y contiene un sensor electroquímico que varía su resistencia interna al estar en contacto con diferentes gases expulsado por sustancias como Amoniaco, Alcohol, Benceno, Humo y Dióxido de carbono. Dentro del sensor se encuentra una bobina de níquel-cromo y que forma parte del sistema calefactor y un revestimiento de dióxido de estaño, el cual es sensible a los gases combustibles, que forma parte de la detección. Una vez se calienta el dióxido de estaño empieza absorber el oxígeno como aire limpio en la superficie, al mismo tiempo que atrae a los electrones del dióxido de estaño dificultando le paso de la corriente a través del sensor. Cuando hay gases en el aire la densidad del oxígeno absorbido por el sensor decrece liberando a los electrones y concediendo que la corriente pase con mayor libertad.

En la ilustración 2 se muestra una gráfica de cómo reacciona la resistencia interna del censo de calidad del aire cuando está en contacto con distintos gases tóxicos y combustibles. En el eje Y se refleja la resistencia que ofrece el sensor y en el eje X se indica la densidad del gas correspondiente:

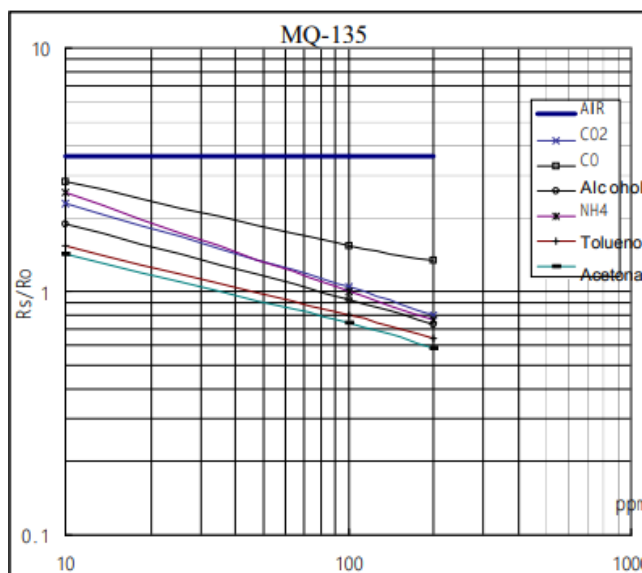


ILUSTRACIÓN 3: GRÁFICA RESISTENCIA/CONTAMINANTE

Por último, se procede a explicar el funcionamiento de la ESP32 dentro del sistema. Una vez todos los sensores explicados anteriormente envían sus datos a través de los pines correspondientes de entrada/salida la ESP32 recibe toda esa información y la almacena en distintas variables internas. Una vez almacenada la información se procede a tratar cada variable de distinta forma según lo requiera ya sea transformándola o tratándola con algún tipo de forma. Esta información tiene dos destinos distintos. Por una parte, la información irá dirigida a la plataforma ThingsBoard, donde estos datos se mostrarán en tiempo real a través de distintas gráficas. Por otra parte, la información se enviará también al *middleware*, un software que hace de intermediario entre el sistema y la base de datos local del mismo, este software se explicará en la siguiente sección.

## 4.2. Funcionamiento del *Middleware*

Para explicar el funcionamiento del *middleware* primero se va a proceder a explicar que es concretamente. Un *middleware* es un software que provee de servicios y funcionalidades comunes a las aplicaciones. Normalmente se lleva a cabo la gestión de los datos, los servicios de aplicaciones y la gestión de las API. Es un software que da asistencia a una aplicación o sistema para interactuar con otras aplicaciones, funciona como una capa de abstracción de software distribuida que se encuentra entre las capas de aplicaciones y las capas inferiores.

En este proyecto el *middleware* es usado como software de comunicación entre dos componentes del sistema que en este caso son la ESP32 por un lado y el Orion Context Broker por el otro lado. La comunicación de estos dos sistemas y el *middleware* se vería como se puede apreciar en la ilustración 3.

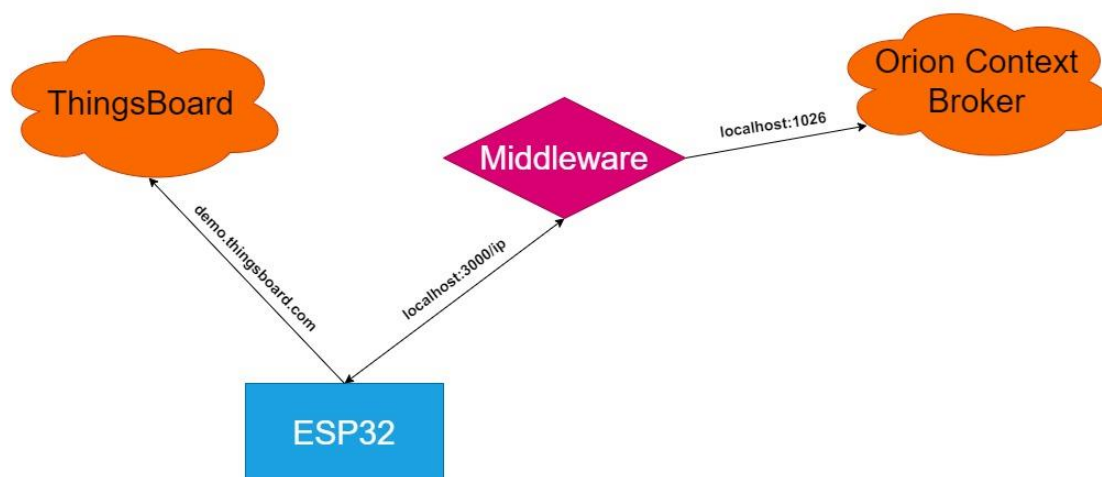


ILUSTRACIÓN 4: ESQUEMA DEL SISTEMA

Por una parte, el *middleware* se conecta a la ESP32 por la dirección "localhost" (en este caso es la dirección 192.168.1.92) con puerto 3000 y la ruta /ip, esta conexión requiere una trama http con una cabecera de tipo Application/json ya que este *middleware* está configurado para recibir solicitudes http de ese tipo. Por otra parte, el *middleware* está conectado al OCB a través de la dirección "localhost" con el puerto 1026 y la ruta /v2/entities/Room5.

El funcionamiento de esta interconexión empieza por la ESP32, donde esta manda los datos recibidos por parte de los sensores del sistema físico al *middleware*, lo hace gracias a unas tramas de código instaladas en su chip junto a las funcionalidades de la librería <HttpClient.h> siguiendo la URL y las rutas almacenadas en sus variables. Una vez la ESP32 establece conexión con el *middleware*, esta empieza a enviar las tramas de datos en formato Json a través de una variable tipo DocumentJson. Este envío de datos se hace a través de una conexión inalámbrica con internet gracias al módulo WiFi del que dispone la ESP32.

Todo este envío de datos por parte de la ESP32 explicado anteriormente no serviría de nada si no se tuviera el *middleware* funcional. Para encender el *middleware* habrá que escribir el comando "npm start" como se muestra en la en la ilustración 4.

```
victo@DESKTOP-OVGF80K MINGW64 ~/OneDrive - Universidad Politécnica de Madrid/Es  

itorio/MIDDLEWARE/LoRa-middleware (main)  

$ npm start  

npm WARN config global '--global', '--local' are deprecated. Use '--location=glo  

bal' instead.  

> LoRa-middleware@0.0.0 start  

> nodemon ./bin/www  

[nodemon] 2.0.16  

[nodemon] to restart at any time, enter 'rs'  

[nodemon] watching path(s): *.*  

[nodemon] watching extensions: js,mjs,json  

[nodemon] starting 'node ./bin/www'  

Listening on port 3000
```

ILUSTRACIÓN 5: INICIALIZACIÓN DEL MIDDLEWARE

Como se puede apreciar en la imagen se ha utilizado un comando “npm”, el cual es un comando perteneciente al software Node JS que se usa para dar soporte y diseñar aplicaciones *network* escalables. Para que este comando funcione en la máquina del usuario, este ha de instalar primero el software NodeJS.

El *middleware* hasta el momento que le llegue una trama de datos queda en espera actuando como si fuera un servidor. Una vez la trama de datos saliente de la ESP32 llega al *middleware*, este lo reconoce al momento y muestra por pantalla la conexión entrante junto a los datos que está recibiendo como se muestra en la ilustración 5.

```
sensor: Room6 updated  

POST triggered coming from IP network  

{  

  deviceName: 'Room6',  

  Temperatura: null,  

  Humedad: null,  

  Luminosidad: 784,  

  C_Aire: 3297  

}  

Mocking deviceExists query for device name: Room6  

Device already registered in Orion  

device: Room6 exists, updating...  

sensor: Room6 updated  

POST /ip 200 996.098 ms - 49
```

ILUSTRACIÓN 6: DATOS ENVIADOS AL MIDDLEWARE

El *middleware* mantiene una plantilla o *layer* la cual designa el tipo de datos que puede recibir, por lo que, si los datos que están llegando al *middleware* no coinciden con esa plantilla, el proceso de envío de datos fallará y el *middleware* lo mostrará por pantalla. La plantilla se muestra en la ilustración 6.

```

var template_ip = {
  "Temperatura": {
    "value": 0,
    "type": "Number"
  },
  "Humedad": {
    "value": 0,
    "type": "Number"
  },
  "Luminosidad": {
    "value": 0,
    "type": "Number"
  },
  "C_Aire": {
    "value": 0,
    "type": "Number"
  },
  "deviceName": {
    "value": "x",
    "type": "String"
  }
};

```

ILUSTRACIÓN 7: PLANTILLA DE DATOS RECIBIDOS POR EL *MIDDLEWARE*

Cuando el *middleware* ha detectado la entrada de nuevos datos por parte de la ESP32, este notifica a Orion Context Broker a través de la URL que el *middleware* tiene internamente guardada, en este caso esa URL es: 192.168.1.92/1026/v2/entities. Posteriormente Orion al percatarse de esta nueva trama de datos, los guarda en su base de datos adyacente que en este caso es MongoDB, los detalles sobre el funcionamiento de Orion se especifican en la siguiente sección.

### 4.3. Funcionamiento de Orion Context Broker

El OCB (Orion Context Broker) es un software el cual aporta una función fundamental en cualquier solución inteligente: administrar la información de contexto, consultarla y actualizarla. Permite la publicación de datos de contexto por entidades como por ejemplo los sensores de manera que los datos de contexto publicados se encuentren disponibles para otras entidades cuyo objetivo es procesar la información también conocidos como consumidores de contexto.

El OCB es un servidor que implementa una API basada en el modelo de información NGSI, que permite realizar diferentes operaciones como registrar aplicaciones de proveedores de contexto, actualizar información de contexto, notificar cuando sucedan cambios en la información de contexto y consultar información de contexto.

Una gran característica a tener cuenta de esta herramienta es la disociación entre productores de contexto y consumidores de contexto. Es decir, que los productores de contexto publican datos sin importar el lugar y la información de los mismos mientras que los consumidores de contexto seleccionan la información en cuestión de su contexto y contenido.

En la ilustración 7 se muestra una imagen de una abstracción gráfica del funcionamiento del OCB.

## API REST (XML & JSON Rendering)

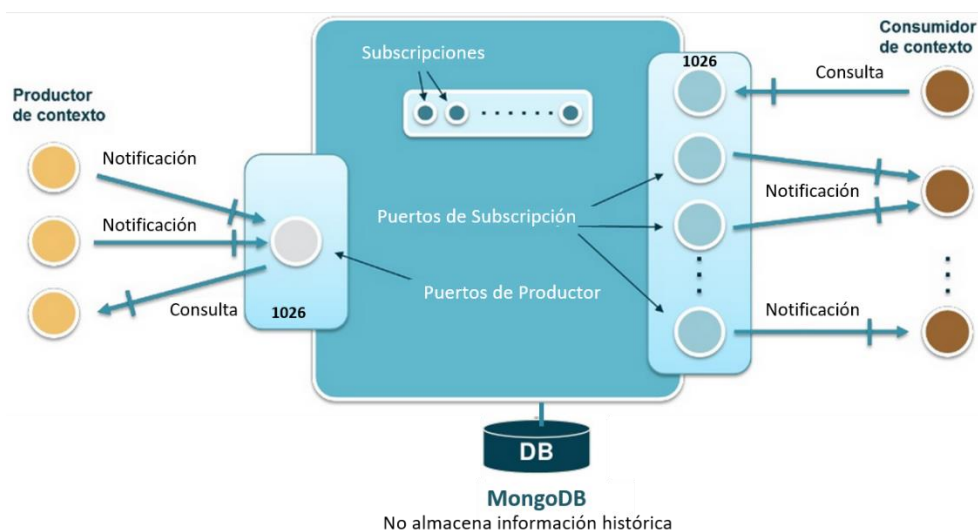


ILUSTRACIÓN 8: FUNCIONAMIENTO ORION CONTEXT BROKER

Una vez explicada y graficada la funcionalidad del OCB, se puede empezar a explicar qué cabida tiene esta herramienta dentro del proyecto que se está desarrollando.

Como se ha explicado anteriormente, la ESP32 manda datos al *middleware* que actúa como un servidor pasivo, este *middleware* está conectado a su vez al puerto 1026 de la red local correspondiente al OCB del sistema. Una vez creado este enlace entre el *middleware* y el OCB a través del puerto 1026, cualquier dato que llegue al *middleware* el OCB lo captará y lo guardará en su base de datos correspondiente que en este caso es MongoDB. Este proceso de recogida de información desde el puerto 1026 hasta la base de datos MongoDB se puede apreciar en la siguiente imagen:

```
{ "id": "Room6", "type": "Room", "C_Aire": { "type": "Number", "value": 3272, "metadata": {} }, "Humedad": { "type": "Number", "value": null, "metadata": {} }, "Luminosidad": { "type": "Number", "value": 780, "metadata": {} }, "Temperatura": { "type": "Number", "value": null, "metadata": {} }, "deviceName": { "type": "String", "value": "x", "metadata": {} } }
```

ILUSTRACIÓN 9: DATOS RECIBIDOS DENTRO DEL OCB

Para que el OCB capte y almacene la información que llega al *middleware* no solo vale con poner el puerto de conexión dentro de la codificación del *middleware*, sino que primero hay que crear una entidad de trabajo para que la información llegue y se almacene en el OCB. Al principio el OCB se encuentra en un estado de “vacío”, no hay ninguna entidad creada por lo que no se pueden cosechar ni almacenar datos del *middleware*, para salir de este estado de vacío primero se crea una entidad introduciendo el siguiente comando donde se especifica la ruta y el tipo de contenido de la nueva entidad. El comando que hay que introducir en la consola para crear esta entidad con los datos que se están recibiendo de los sensores se muestra en la ilustración 9.

```
curl localhost:1026/v2/entities -s -S -H 'Content-Type: application/json' -d @- <<EOF
{
  "id": "Room6",
  "type": "Room",
  "temperatura": {
    "value": 0,
    "type": "Number"
  },
  "humedad": {
    "value": 0,
    "type": "Number"
  },
  "Luminosidad": {
    "value": 0,
    "type": "Number"
  },
  "C_Aire": {
    "value": 0,
    "type": "Number"
  },
  "deviceName": {
    "value": "x",
    "type": "String"
  }
}
EOF
```

ILUSTRACIÓN 10: COMANDO DE CREACIÓN DE ENTIDAD

Como se puede apreciar en la imagen se está declarando una nueva entidad la cual se encuentra en la red local con puerto 1026 y ruta “/v2/entities”, dentro de esta ruta se crea otra ruta con el nombre de “Room6” que va a ser el nombre e identificador de la nueva entidad creada. Después de declarar el identificador y tipo de la entidad se prosigue con la declaración de las distintas variables que son esperadas para ser detectadas y almacenadas por el OCB. Como las variables esperadas por el sistema son de tipo decimal, lo más normal es que estas variables fueran declaradas en el comando como tipo “float”, pero el OCB tiene un sistema interno de datos un tanto especial y el OCB en vez de declarar estos datos como tipo “float” lo hace como tipo “Number”.

Una vez se tiene el comando listo según los requisitos y variables con los que se quiere crear la entidad, se procede a enviar el comando e inmediatamente el OCB crea la entidad y ya se encuentra disponible para interactuar con ella a través del *middleware*. Lo último que queda por hacer es especificar la ruta guardada en la variable que apunta a la dirección de la entidad creada en el OCB, que es la siguiente: <http://localhost:3000/v2/entities/Room6>.

## 4.4. Funcionamiento de ThingsBoard

Thingsboard es una herramienta de código abierto online que permite un ágil desarrollo, gestión y escalado de proyectos de IoT. El objetivo de esta plataforma es proporcionar una solución local o en la nube de los proyectos IoT que habilita la infraestructura del lado del servidor para sus correspondientes aplicaciones IoT.

Esta plataforma tiene un amplio abanico de usos dentro de la industria del internet de las cosas, desde una simple implementación de una entidad con la que poder interactuar hasta un complejo sistema de interacción implementado en su totalidad dentro de la nube de la herramienta.

A pesar de la gran cantidad de funcionalidades que puede ofrecer esta herramienta, en este proyecto se ha decidido que la plataforma ThingsBoard va a ser usada como herramienta para graficar los datos mandados por parte de los sensores.

Para poder graficar los datos del sistema en la herramienta ThingsBoard primero hay que realizar unos pasos previos para montar el sistema de graficado. En primer lugar, se tendrá que crear un dispositivo al que se le va a llamar “ESP32” dentro de la gestión de los dispositivos de la herramienta, la creación se haría como se muestra en la ilustración 10.

**Agregar nuevo dispositivo**

1 Detalles del dispositivo 2 Credenciales Optional 3 Cliente Optional

Nombre \*  
ESP32

Etiqueta  
ESP32

☒ Seleccionar un perfil existente ☐ Crear un nuevo perfil de dispositivo

Perfil de dispositivo \*  
default

☐ Es gateway

Descripción

ILUSTRACIÓN 11: AÑADIR DISPOSITIVO THINGSBOARD

Una vez se tiene el dispositivo creado simplemente hay que ir al apartado de “Paneles” y ya se pueden empezar a crear las plantillas de las gráficas que se van a utilizar en el proyecto.

Las plantillas gráficas usadas en el panel de este proyecto se muestran en la ilustración 11.

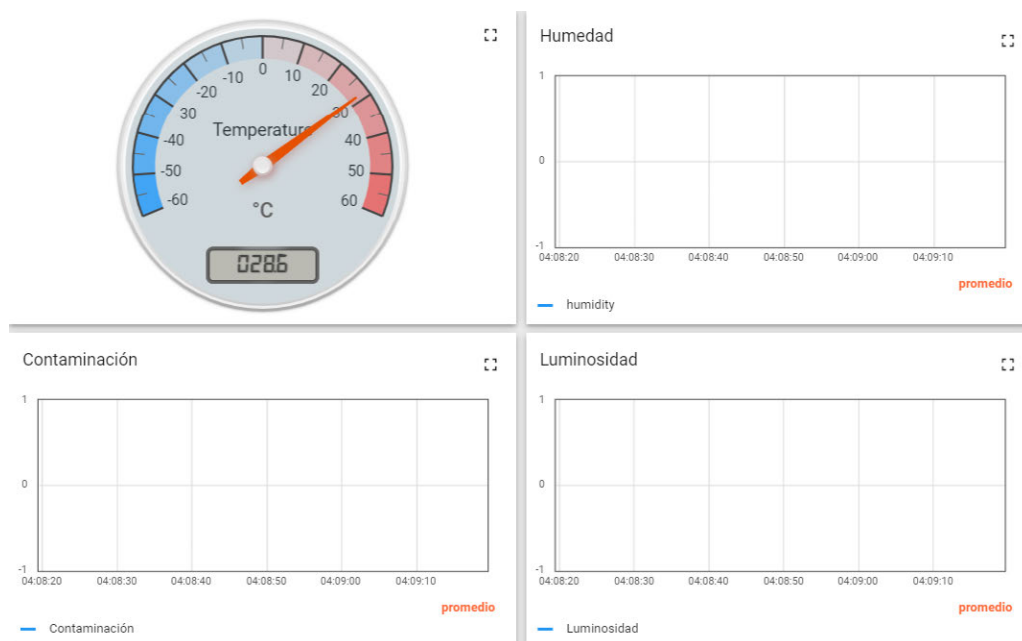


ILUSTRACIÓN 12: MOSTRAR DATOS THINGSBOARD

Como se puede apreciar en la imagen de arriba, se ha seleccionado que para la luminosidad, contaminación y humedad se utilice un gráfico que muestre un histórico en un intervalo de tiempo de 30 segundos de lectura de datos. En cuanto a la temperatura se ha decidido mostrar el dato dentro de una plantilla estática y visual.

Una vez se tienen las plantillas el panel ya está casi listo, pero para que muestren algún dato y no se queden en blanco sin mostrar nada habrá que especificar el valor interno que tendrán que leer, para ello simplemente hay que meterse en la configuración de la gráfica y dentro del apartado “datos”, seleccionar la entidad y la variable que se quiere leer. En la ilustración 12 se muestra un ejemplo de cómo está configurada la gráfica referente a la variable de la humedad.

|    | Tipo    | Parámetros     |
|----|---------|----------------|
| 1. | Entidad | Humedad        |
|    | Filtro  | Últimas claves |

ILUSTRACIÓN 13: CONFIGURACIÓN DE GRÁFICA

Especificando los datos que se reciben en cada gráfica el panel ya estaría terminado, ahora solo queda la parte referente a la recepción de los datos generados del sistema desarrollado en este proyecto.

La ESP32 durante la fase de gestión y envío de datos, no solo envía información al *middleware*, sino que en la programación interna de la placa se han codificado varias líneas referentes al envío de datos a la plataforma ThingsBoard. Para ello, los desarrolladores de esta plataforma han creado una librería de Arduino llamada <ThingsBoard.h> la cual gestiona todas las comunicaciones necesarias entre el sistema IoT y la herramienta.

Para poder enviar datos, el sistema no solo necesita saber la URL a la que se envían los datos (que en este caso es <https://demo.thingsboard.io/>), sino que también es necesario un “Token”, es decir, una clave que se usa para enviar datos a una entidad concreta. La clave de la entidad creada en el ThingsBoard de este proyecto se muestra en la ilustración 13.

Credenciales del dispositivo

Tipo de credenciales  
Access token

Tóken de acceso \*

VNbTk60funcwJz2REj11

Cancelar Guardar

ILUSTRACIÓN 14: TOKEN DE ENTIDAD

Una vez ya se dispone de la URL y la clave de la entidad, simplemente se codifican unas líneas de envío de datos provistas por la biblioteca de ThingsBoard y los datos empiezan a llegar a la plataforma teniendo así una herramienta para graficar en tiempo real y accesible desde cualquier parte del mundo.

## 5. Implementación Hardware

En este apartado se va a explicar todo lo relacionado con la parte hardware del sistema, desde los dispositivos electrónicos componentes hasta las conexiones de los mismos a lo largos del sistema IoT.

### 5.1. Dispositivos del Sistema

Se van a explicar los diferentes dispositivos electrónicos usados en el sistema. Se redactará qué es cada dispositivo, para qué es usado y qué función tiene el dispositivo en el sistema implementado:

- **Módulo ESP32**

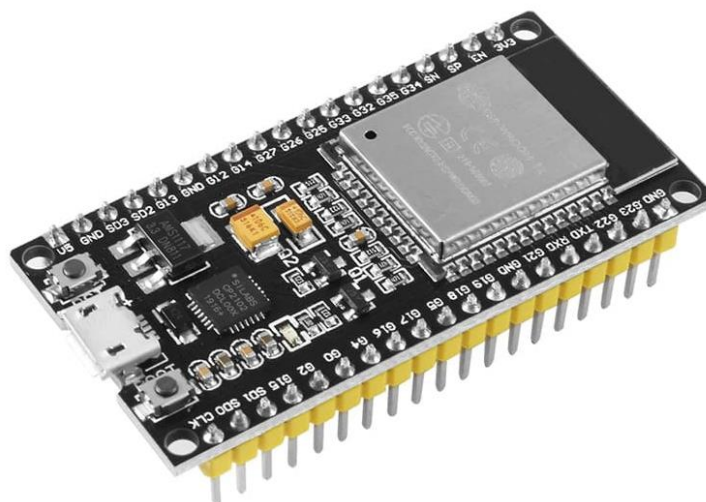


ILUSTRACIÓN 15: MÓDULO ESP32

El módulo ESP32 implementa unas prestaciones inalámbricas Bluetooth/Wi-Fi como una solución certificada e integrada. Además de proporcionar este servicio inalámbrico, posee un procesador integrado con interfaces para poder conectarse con diversos periféricos. Se le denomina ESP32 a la familia de chips SoC (*System on Chip*), este modelo de chip emplea un microprocesador Tensilica Xtensa LX6 en sus variantes simples y doble núcleo e incluye interruptores de antena, amplificador de potencia, filtros, y módulos de administración de energía.

Este módulo es frecuentemente utilizado en sistemas IoT de bajo coste (debido a que su precio en el mercado no es mayor a 10€) y con una bajo demanda de energía. El módulo actúa de sistema procesador de los datos recibidos por los diferentes dispositivos conectados al sistema IoT. Este tipo de módulo ha adquirido gran popularidad debido a su fácil portabilidad, accesibilidad y capacidad de tener una conexión a la red de forma inalámbrica.

En este proyecto, el módulo ESP32 será la placa principal del sistema, que procesará y enviará los datos recogidos por los sensores al sistema de almacenamiento. Este módulo ha sido determinado como el idóneo para desarrollar este proyecto ya que, a pesar de su bajo coste, posee las suficientes funcionalidades como para no necesitar un módulo procesador de una

gama mayor. Además, el hecho que posea servicios de conexión inalámbrica es determinante ya que hace al sistema bastante más independiente y portátil, por lo que se podrá trasladar el sistema a diferentes puntos geográficos sin necesidad de tener que depender de ningún tipo de cable ya sea de red o de energía.

Los diferentes dispositivos que conforman el sistema se van a conectar a la placa ESP32 mediante los pines de conexión. El modo de funcionamiento de cada pin se configurará en el código de funcionamiento del sistema. En la ilustración 15 se muestra un esquema de los pines de conexión que ofrece la placa ESP32.

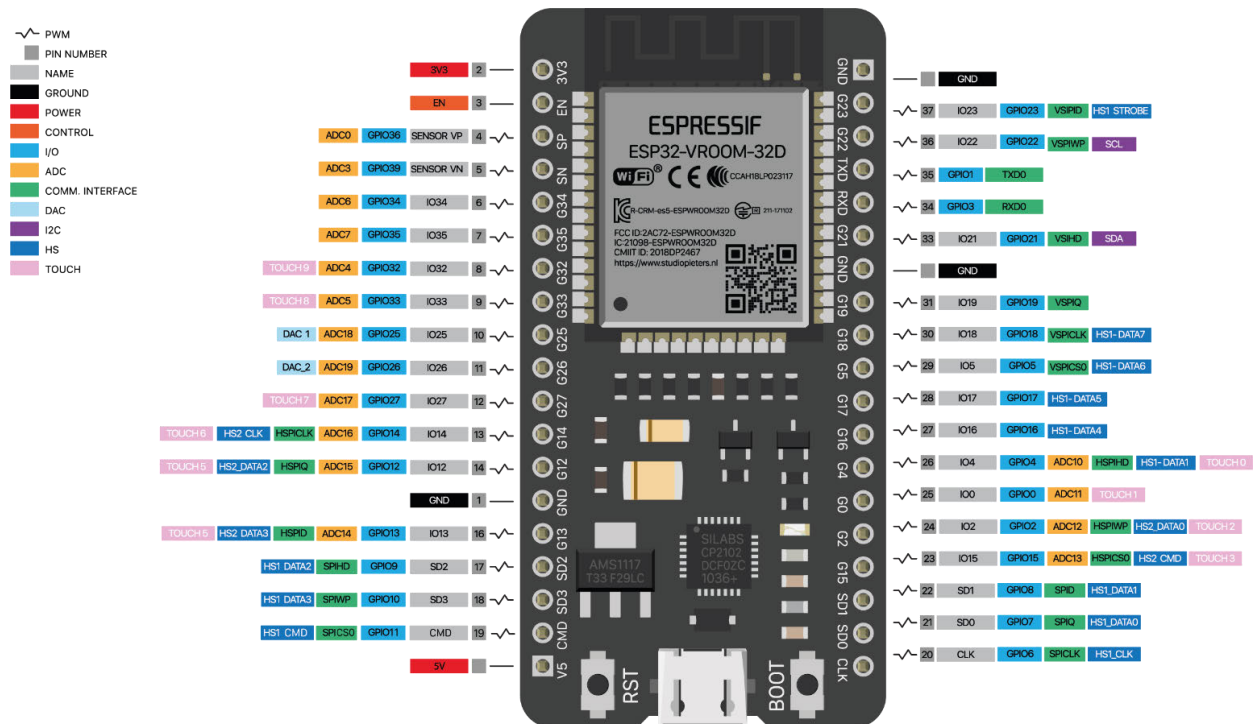


ILUSTRACIÓN 16: ESQUEMA DE PINES ESP32

En esta última imagen se pueden apreciar los diferentes pines de conexión que tiene el módulo ESP32 como pueden ser las entradas de alimentación, la tierra, los pines ADC... En especial destacan los pines de entrada y salida que son los marcados con la abreviatura 'GPIO' (*General Purpose Input/Output*), en estos pines es donde se conectarán los sensores y periféricos que se utilizarán en el sistema.

- **Sensor DHT11:**

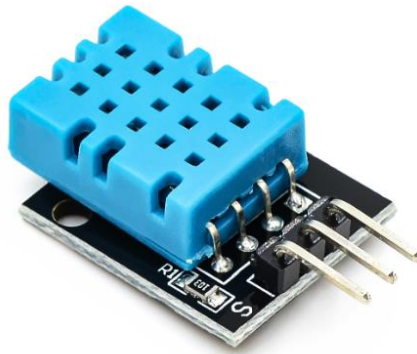


ILUSTRACIÓN 17: SENSOR DHT11

El DHT11 es un sensor digital que tiene como finalidad detectar tanto la temperatura como la humedad del lugar donde se le coloca. Este sensor consume poca energía y es económicamente accesible. El dispositivo integra un sensor capacitivo de humedad y un termistor para medir el aire del entorno donde se encuentra, y muestra los datos obtenidos por los sensores a través de una señal digital. Es usualmente utilizado en proyectos académicos, en dispositivos como aires acondicionados y en monitoreo ambiental y agrícola.

En este proyecto se usará este sensor como medidor de datos meteorológicos usando tanto la medición de temperatura como la de humedad. Una vez tomados los datos con este sensor, se envían al *middleware* del sistema a través de la ESP32 y finalmente son guardados en la base de datos del sistema. Adicionalmente, el sistema mostrará estos datos a través de la plataforma ThingsBoard.

Las conexiones de este sensor son las mostradas en la ilustración 17.

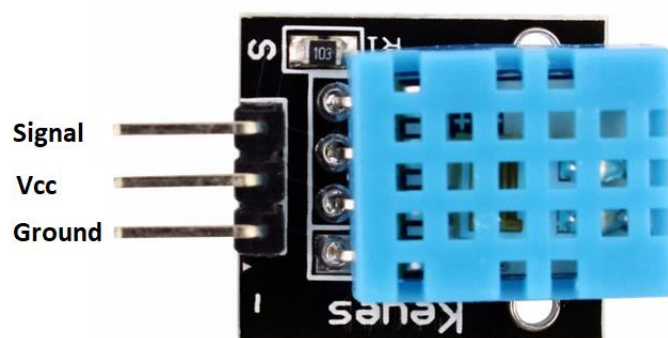


ILUSTRACIÓN 18: CONEXIONES SENSOR DHT11

- Signal: Pin de salida de los datos detectados por el sensor.
- Vcc: Pin de entrada de alimentación (5V).
- Ground: Pin de toma de tierra.

- **Fotoresistor**

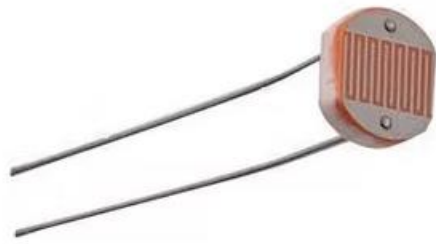


ILUSTRACIÓN 19: FOTORESISTOR

Este componente es una foto-resistencia capaz de medir la intensidad lumínica. Esto es posible gracias a que es un componente pasivo que reduce la resistencia al paso de la electricidad a medida que la luz choca con el material fotosensible del componente, es decir que cuanto más intensa sea la luz que choca con la foto-resistencia, menor será la resistencia al paso de la electricidad.

Este sensor se utilizará en el proyecto para medir la intensidad lumínica de los distintos puntos de prueba en los que se pretende probar el sistema, como líneas futuras se pensaría en medir la intensidad lumínica durante un día entero en diferentes puntos geográficos y comparar los resultados entre sí. Este componente mantiene un gasto de energía despreciable y su coste en el mercado es bastante asequible, lo que le hace ideal para este proyecto.

Respecto a las conexiones de este componente, se va a montar de tal forma que una de las patas esté conectada a la alimentación de 5 voltios, y la otra pata estará conectada a un pin de datos de entrada de la ESP32, esta última pata también estará conectada a una resistencia de 1KOhm la cual conecta con la tierra (GND).

- **Sensor MQ-135**



ILUSTRACIÓN 20: SENSOR MQ-135

Este sensor es un detector de calidad del aire que integra un sensor electroquímico que varía su resistencia al estar en contacto con gases que desprende el amoníaco, el alcohol, el benceno, el humo y el dióxido de carbono. El módulo del sensor permite conectar el sensor integrado con algún tipo de placa de desarrollo, como la ESP32 en el caso de este proyecto.

Este sensor se utilizará en el proyecto para medir la calidad del aire del entorno donde se posicione el sistema. Gracias a este sensor se detectará si existen gases nocivos alrededor y registrarlo en la base de datos del sistema. Los datos registrados por parte de este sensor podrán visualizarse además en la herramienta ThingsBoard en tiempo real.

En la ilustración 20 se muestran las conexiones de este sensor.

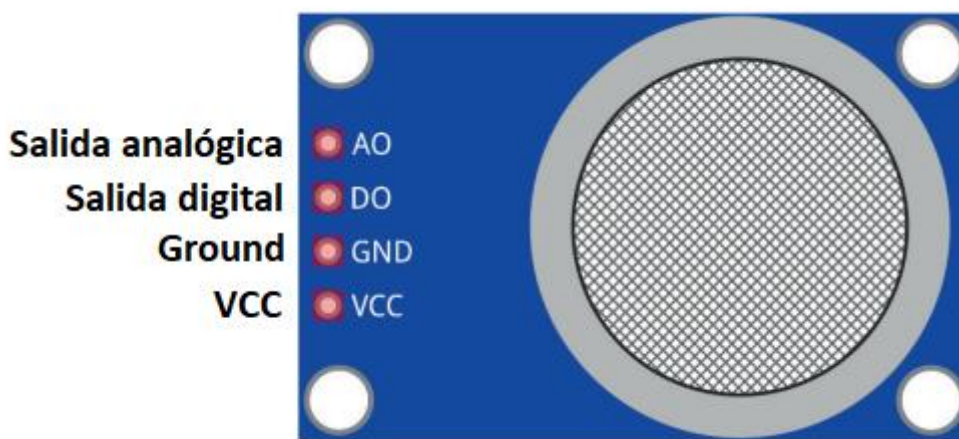


ILUSTRACIÓN 21: CONEXIONES SENSOR MQ-135

- o AO: Pin de salida de los datos analógicos detectados por el sensor.
- o DO: Pin de salida de los datos digitales detectados por el sensor.
- o Vcc: Pin de entrada de alimentación (5V).
- o Ground: Pin de toma de tierra.

En este proyecto se utilizará como pin de salida únicamente el pin analógico (AO).

## 5.2. Esquema del sistema

En la ilustración 21 se muestra la disposición del sistema físico contando con la ESP32 y los distintos componentes.

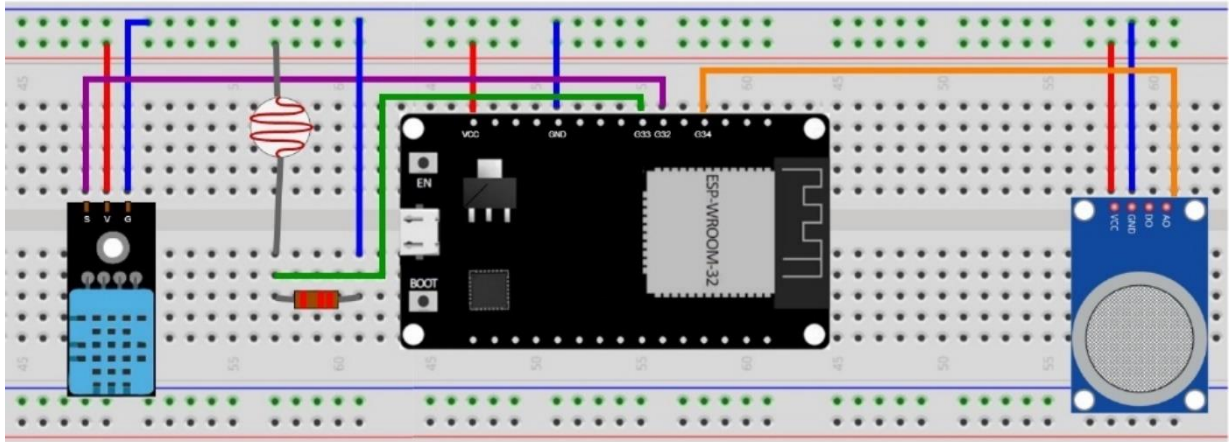


ILUSTRACIÓN 22: RECREACIÓN DEL SISTEMA FÍSICO

Las conexiones de los componentes con la ESP32 son las siguientes:

- DHT-11:
  - o Vcc - Genérico Vcc
  - o Gnd - Genérico Ground
  - o Signal – GPIO32
  
- MQ-135:
  - o Vcc - Genérico Vcc
  - o Gnd - Genérico Ground
  - o AO – GPIO34
  
- Fotorresistor:
  - o OUT – GPIO33

En la ilustración 22 imagen se muestra como luce el sistema físico del proyecto en la realidad y se indica la disposición de los diferentes sensores.

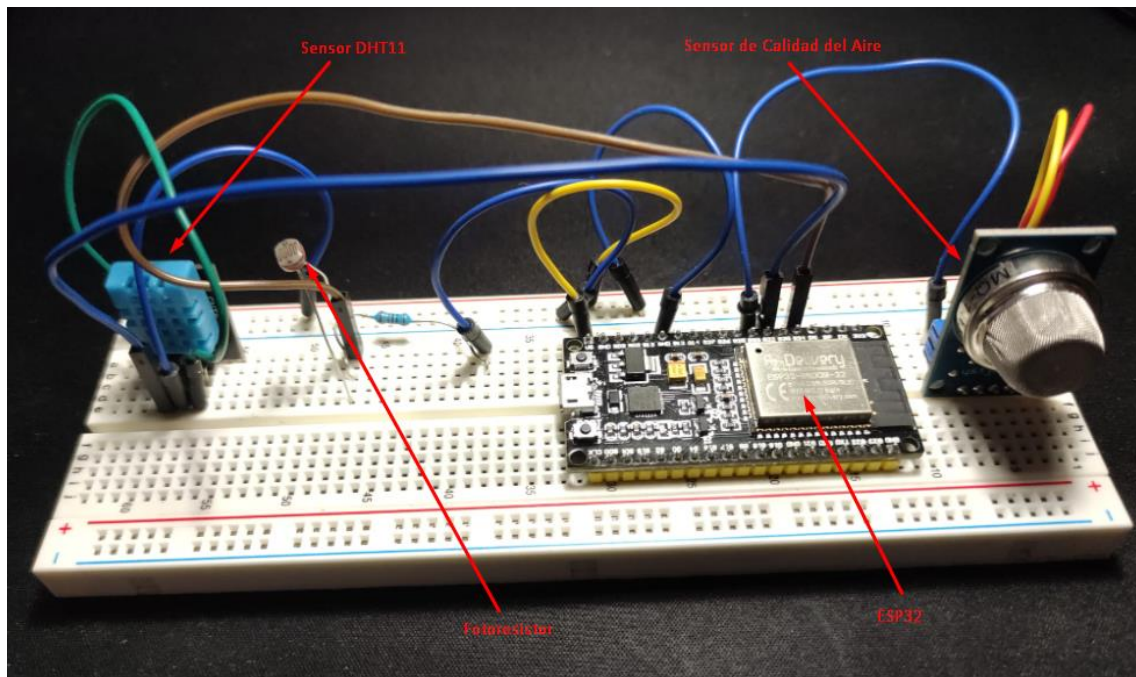


ILUSTRACIÓN 23: SISTEMA FÍSICO REAL

## 6. Resultados

Los resultados que se han obtenido en este proyecto se reparten en diferentes partes del sistema. Se van a exponer los resultados obtenidos en los sensores tanto en la herramienta ThingsBoard, la cual se ha usado para graficar los datos, como en el OCB y la base de datos referente MongoDB.

### 6.1. Resultados del Orion Context Broker

Para mostrar los resultados con la herramienta OCB se va a enseñar el proceso que sigue una muestra de datos desde la detección hasta el almacenamiento. Esta muestra de datos está tomada dentro de una habitación.

En primer lugar, como se puede ver en la ilustración 23 el sistema recoge las muestras de datos y las enseña a través del monitor serie.

```
HTTP Response code: 200
Temperatura:
29.40
Humedad:
35.00
Intensidad de Luz:
948
Contaminación:
3249
```

ILUSTRACIÓN 24: MUESTRA DE DATOS EN EL MONITOR SERIE

En la imagen se puede apreciar como el sistema se conecta al *middleware* ya que está mostrando la respuesta http que este da. Cuando el *middleware* devuelve la petición http con código 200 quiere decir que la conexión con este ha sido exitosa y los datos han sido tomados por el OCB satisfactoriamente.

En la ilustración 24 se muestra cómo se aprecia la recepción de los datos por parte del *Middleware*.

```
POST triggered coming from IP network
{
  deviceName: 'Room6',
  Temperatura: 29.39999962,
  Humedad: 35,
  Luminosidad: 948,
  C_Aire: 3249
}
Mocking deviceExists query for device name: Room6
Device already registered in Orion
device: Room6 exists, updating...
sensor: Room6 updated
POST /ip 200 101.814 ms - 49
```

ILUSTRACIÓN 25: MUESTRA DE DATOS EN EL MIDDLEWARE

En esta captura del log del *middleware*, se pueden apreciar distintos mensajes que indican como se está comportando y funcionando el sistema.

En primer lugar, el *middleware* detecta la trama y automáticamente la muestra por el *log* de la consola junto al mensaje “POST triggered...”. Una vez muestra estos datos el *middleware* procede a simular el encolamiento de datos con nombre de identidad “Room6” y el OCB detecta que esta entidad ya ha sido creada y no tiene que crear una nueva dentro de la base de datos. Acto seguido, el OCB empieza a actualizar la base de datos con los datos que han llegado al *middleware*.

Una vez todos los datos han sido actualizados dentro del OCB, el *middleware* muestra por pantalla que la conexión con la traza http *POST* ha tenido éxito, mostrando el código http “200” y la duración de todo el proceso de conexión y actualización en milisegundos.

Por último, en la ilustración 25 se muestra cómo se almacenan los datos en la base de datos del OCB.

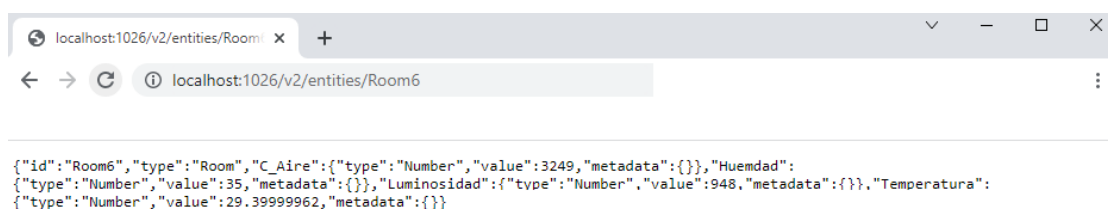


ILUSTRACIÓN 26: MUESTRA DE DATOS ALMACENADA EN EL OCB

Como se puede ver en la imagen una de las formas de acceder a la base de datos integrada del OCB es yendo a su dirección local URL correspondiente, que en este caso es: <http://localhost:1026/v2/entities/Room6>. Dentro de esta URL se va a mostrar el contenido de los datos enviados a través del *middleware* a la entidad “Room6” (entidad usada para el envío de los datos en este proyecto). Se puede apreciar cómo se muestran las diferentes variables de datos junto al tipo de variable que son, el nombre y el valor recibido por parte del sistema y el *middleware*.

En cuanto a la muestra de datos, ha sido tomada dentro de un habitáculo cerrado con una luminosidad media-baja (la luminosidad es inferior a 1000 puntos) ya que se la muestra se ha tomado a las 21:00. La humedad es algo superior a la del exterior ya que, al estar el habitáculo cerrado, se genera más humedad. La contaminación en el aire es apenas notable con 3200 puntos frente a los 7000 puntos a los que habría que llegar para que se considerara que el aire del habitáculo es peligroso para la salud. Por último, la temperatura con 30°C es algo elevada a pesar de estar en época de verano, esto es debido al calor que expulsan los dispositivos dentro del habitáculo.

## 6.2. Resultados de ThingsBoard

La recepción y muestra de los datos en ThingsBoard, como se ha visto en otros apartados de esta memoria, sigue un proceso mucho más simple comparado con el del apartado anterior.

De nuevo se va a utilizar un escenario dentro de un habitáculo cerrado para la muestra de datos.

Primero los datos son mostrados por el monitor serie, como en el caso anterior:

```
Connecting to: demo.thingsboard.io with token VNbTk60funcwJz2REj11

HTTP Response code: 200
Temperatura:
29.90
Humedad:
37.00
Intensidad de Luz:
1085
Contaminación:
3414
```

ILUSTRACIÓN 27: MUESTRA DE DATOS EN EL MONITOR SERIE

A diferencia del caso anterior se puede apreciar como el sistema según comienza el ciclo de muestreo de datos, empieza a establecer la conexión con la plataforma ThingsBoard. Además, el sistema especifica qué plataforma se está usando (ThingsBoard tiene varias plataforma con diferentes finalidades), mostrando la URL de la misma seguido del “token” de la entidad a la que se le están enviando los datos. Acto seguido el sistema muestra por este monitor serie los datos que han sido recogidos de los sensores.

Los datos recogidos son enviados a ThingsBoard junto con el “token” de la entidad. Una vez llegan los datos a ThingsBoard, este los trata y los representa de la siguiente forma:

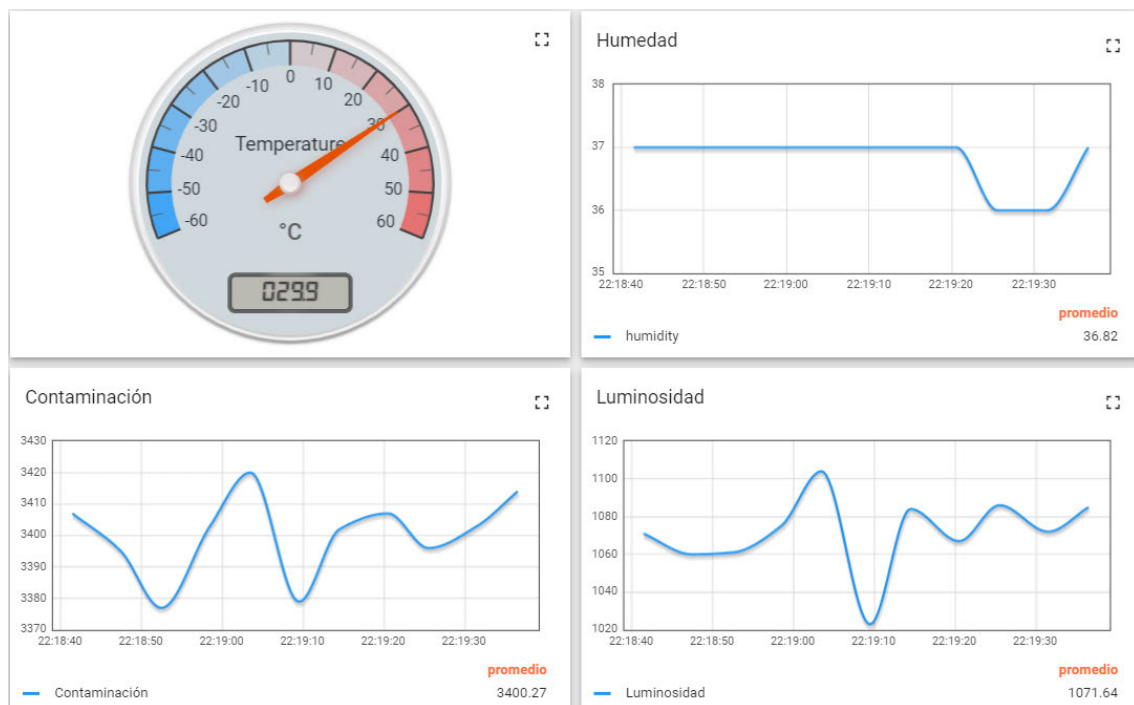


ILUSTRACIÓN 28: MUESTRA DE DATOS EN THINGSBOARD

Como se puede apreciar en la imagen, la herramienta para graficar datos de ThingsBoard permite ver como varían las mediciones de los sensores a lo largo del tiempo. Incluso, estos datos pueden ser supervisados y accedidos desde cualquier parte del plante y en tiempo real, dándole así una gran funcionalidad e independencia al sistema.

## 7. Aspectos éticos, legales y medioambientales

Hoy en día cualquier proyecto que requiera de la creación de un sistema involucra una carga medioambiental tras su desarrollo, y este proyecto no va a ser la excepción. Aunque la huella medioambiental registrada por este proyecto es mínima, es necesario mencionarla para tomar nota y no dejar de lado el objetivo común que tienen todas las personas de preservar este planeta.

La huella de carbono que puede registrar este proyecto como se ha mencionado es mínima, empezando por la más significativa, que es el gasto energético que genera el sistema físico mientras está encendido. El dispositivo necesita una pequeña cantidad de energía para funcionar que apenas es apreciable, pero al tener que estar tanto tiempo conectado para hacer las mediciones pertinentes este gasto energético se vuelve algo significativo. Aún así gracias a la tecnología innovadora de las últimas versiones de las placas ESP, ahora traen un modo de funcionamiento llamado *Deep-sleep mode* el cual según el funcionamiento requerido de la placa puede disminuir el consumo de esta de forma drástica. El modo de funcionamiento mínimo dentro del *Deep sleep* es el modo “*only RTC*” en el que el único sistema interno de la ESP32 funcional es el reloj interno en tiempo real, con este modo el consumo eléctrico llega a ser de 10 $\mu$ A.

Por otra parte, se tiene el gasto de los dispositivos de los que depende el sistema físico, como por ejemplo el gasto por parte de las máquinas anfitrionas de los servidores necesarios para que el sistema funcione correctamente. Por una parte, la máquina local en la que se tenga instalado el servidor *middleware* va a gastar de media unos 2200kWh de media al día dependiendo del uso que se le dé al sistema. También hay que tener en cuenta el consumo de los servidores que tienen los servidores de la plataforma ThingsBoard, el gasto aproximado de eso servidores puede incluso llegar a cuadruplicar el gasto de la máquina local mencionada dependiendo del tipo de servidor que se utilice.

Los sensores utilizados en el sistema mantienen un perfil de bajo consumo energético como se menciona en la sección de “Implementación Hardware”. Se han seleccionado minuciosamente estos sensores para que sean útiles en cuanto al objetivo del proyecto, y además, tengan un consumo energético mínimo debido a la gran cantidad de horas que el sistema necesita estar funcionando.

Todo este gasto eléctrico mencionado anteriormente es algo muy a tener en cuenta ya que uno de los causantes principales del cambio climático y de la huella de carbono es la generación de electricidad. Por ello hay que desarrollar sistemas IoT en los que el consumo eléctrico no sea muy elevado ya que en unos años habrá considerablemente más sistemas relacionados con el internet de las cosas y si estos sistemas tienen un alto consumo energético podrían llegar a causar un desastre energético en el futuro.

Por ello lo más importante a estas alturas es evitar tener una gran influencia en la huella de carbono causada en el planeta con la creación de estos sistemas. Para ello los sistemas IoT tienen importantes usos como los siguientes:

- Aumentar la eficiencia energética
- Mejorar la eficiencia de la iluminación
- Mejorar la eficiencia de calefacción y refrigeración
- Evolucionar a un entorno de trabajo sin papel

- Implementar el *check-out* automatizado
- Optimizar y automatizar la cadena de suministro
- Reducir la logística innecesaria

En cuanto a los aspectos éticos en los que puede afectar este proyecto no se ha encontrado ninguno. Los aspectos éticos tienen relación con el tratamiento de datos e información sensible de las personas, en este proyecto no se interacciona con usuarios ni con sus datos en ningún aspecto. Por otra parte, estos aspectos éticos también engloban el uso que se le da a estos sistemas, teniendo que llegar a tener que valorar lo ético que es el objetivo que se le da a estos proyectos IoT.

En cuanto a los aspectos legales en los que pueda influir este proyecto, al igual que con los aspectos éticos, apenas hace huella en este campo más allá de regulaciones legales de los dispositivos que utiliza el sistema.

## 8. Conclusiones

### 8.1. Desafíos encontrados

Durante el desarrollo de este proyecto se han encontrado numerosos desafíos y obstáculos que han dificultado el avance del mismo. Uno de los primeros desafíos ha sido plantear un proyecto de este calibre de la nada, un proyecto con el que se cierra un ciclo académico y en el que se tiene que demostrar parte de los conocimientos que se han adquirido durante la carrera, lo que ha supuesto la toma de una gran y difícil decisión como epílogo de este periodo académico.

Otro de los desafíos del proyecto ha sido el tener que familiarizarse con una nueva tecnología como es Fiware, teniendo una amplia funcionalidad y herramientas ha sido complicado ubicar la funcionalidad de este software dentro del desarrollo del proyecto. El hecho de tener que adaptar un sistema programado con Arduino, a un sistema externo dependiente de Docker y JavaScript.

La falta de conocimientos sobre las herramientas de Docker y Fiware han dificultado mucho el avance del proyecto. Además, muchas de las ideas iniciales del mismo han tenido que ser descartadas debido a la desactualización de la documentación y software sobre el cual se quería trabajar. Finalmente simplificando estas ideas iniciales y mezclándolas se ha podido seguir adelante con este proyecto.

En cuanto a la parte hardware del proyecto, ha sido difícil establecer un objetivo claro sobre las mediciones de los sensores. Debido a que el grueso del proyecto ha sido desarrollar todo el sistema que hace posible el envío, conexión y funcionalidad tanto del *Middleware* como del Orion Context Broker con el módulo ESP32, llegar a una decisión de cómo orientar la funcionalidad del sistema a la vida real ha sido complicado a pesar de que no fuera la parte más fundamental del proyecto.

Por último, uno de los retos más complicados que se han tenido que afrontar en el proyecto ha sido la falta de tiempo general que se ha desembocado por diferentes situaciones que han ocurrido durante el desarrollo del mismo. El hecho de tener poco tiempo para desarrollar un proyecto como es un trabajo de fin de grado ha sido bastante desafiante ya que esta situación ha forzado a tomar decisiones que no estaban planteadas durante en un inicio.

### 8.2. Conocimientos adquiridos

Todos los desafíos mencionados en el apartado anterior no solo han supuesto un obstáculo o un reto, sino que también han supuesto una situaciones de las cuales se ha podido adquirir nuevos conocimientos.

Para empezar, el hecho de tener que trabajar con una nueva herramienta como es Docker ha supuesto un gran aprendizaje sobre las funcionalidades de esta herramienta, pudiendo así utilizarla durante futuros proyectos relacionados con el campo del IoT y con otros campos ya sea para crear pequeñas instancias de servidores, o incluso crear aplicaciones funcionales.

El hecho de tener que lidiar con un *middleware* programado en lenguaje Java Script ha supuesto otra fuente de aprendizaje, ya que se partía de un conocimiento nulo del mismo y una vez finalizado este proyecto se ha adquirido cierto grado de conocimiento del lenguaje de programación.

Por supuesto, el desarrollar otro proyecto más en Arduino fortalece los conocimientos adquiridos de este lenguaje, y además favorece el futuro desarrollo referente al hardware programado con Arduino.

Por último, el tener que desarrollar un proyecto de este calibre en solitario ha sido algo bastante desafiante que ha supuesto un cierto incremento de madurez y seriedad en este tipo de proyectos. El hecho de depender de tus propios conocimientos y aptitudes ha sido un hecho positivo para futuros proyectos laborales.

### 8.3. Líneas futuras

Como se ha comentado antes, el reducido tiempo que se ha tenido para desarrollar este proyecto ha causado que se llegue al final del mismo sin cumplir alguno de los hitos iniciales. Además, durante el desarrollo del proyecto y según surgían las ideas, se han pensado en diferentes funcionalidades que darían un salto de calidad al proyecto dentro de la gran funcionalidad que ya tiene.

Los hitos futuros de este proyecto podrían ser los siguientes:

- Crear una serie de alertas a través de Telegram y con ayuda de las funcionalidades de ThingsBoard para notificar al usuario cuando alguna de las variables medidas supera algún tipo de límite.
- Ampliar la gama de sensores para la detección de factores ambientales del sistema, como por ejemplo un sensor de viento para medir la velocidad del mismo o un sensor de presión atmosférica.
- Registrar información ambiental en distintos puntos geográficos y comparar las diferencias de los datos obtenidos.
- Ampliar las funcionalidades Fiware del sistema utilizando herramientas como Quantum Leap, Grafana (integrado en Docker) o Postman.
- Reemplazar el módulo ESP32 por un módulo que ofrezca más funcionalidades y sea óptimo en cuanto a su funcionamiento.

## 9. Bibliografía

- [1] Madakam, S., Lake, V., Lake, V., & Lake, V. (2015). Internet of Things (IoT): A literature review. *Journal of Computer and Communications*, 3(05), 164.
- [2] Fox, J., Suda, K., Facca, F. M., Lo Grasso, F., (2020). GitHub - FIWARE/tutorials.Time-Series-Data: FIWARE 304: Querying Time Series Data (QuantumLeap). GitHub. <https://github.com/FIWARE/tutorials.Time-Series-Data>
- [3] T. (2013). fiware-orion/install.md at master · telefonicaid/fiware-orion. GitHub. <https://github.com/telefonicaid/fiware-orion/blob/master/doc/manuals/admin/install.md>
- [4] ESP32: Connecting to a WiFi network. (2022, 18 enero). Techtutorialsx. <https://techtutorialsx.com/2017/04/24/esp32-connecting-to-a-wifi-network/>
- [5] NodeJS - Setup a Simple HTTP Server / Local Web Server. (2016, 22 junio). Jason Watmore's Blog. <https://jasonwatmore.com/post/2016/06/22/nodejs-setup-simple-http-server-local-web-server>
- [6] Abuse, S. (2020, 29 abril). Cómo crear un servidor web en Node.js con el módulo HTTP. DigitalOcean Community. <https://www.digitalocean.com/community/tutorials/how-to-create-a-web-server-in-node-js-with-the-http-module-es>
- [7] Fundación Innovación Bankinter. (2022, 22 marzo). Las 6 tecnologías que están potenciando el internet de las cosas. [https://www.fundacionbankinter.org/noticias/las-6-tecnologias-que-estan-potenciando-el-internet-de-las-cosas/?\\_adin=11551547647](https://www.fundacionbankinter.org/noticias/las-6-tecnologias-que-estan-potenciando-el-internet-de-las-cosas/?_adin=11551547647)
- [8] Hernandez, R. (2019, 9 diciembre). ¿Qué es la tecnología LoRa y por qué es importante para IoT? The Things Network. <https://www.thethingsnetwork.org/community/santa-rosa/post/que-es-la-tecnologia-lora-y-por-que-es-importante-para-iot>
- [9] Doménech, F. (2020, 10 abril). ¿Quién inventó el primer ordenador? OpenMind. <https://www.bbvaopenmind.com/tecnologia/visionarios/quien-invento-el-primer-ordenador/#:%7E:text=El%20primer%20ordenador%20que%20s%C3%AD,de%20la%20bomba%20de%20hidr%C3%B3geno>
- [10] Colaboradores de Wikipedia. (2022, 8 junio). Circuito integrado. Wikipedia, la enciclopedia libre. [https://es.wikipedia.org/wiki/Circuito\\_integrado](https://es.wikipedia.org/wiki/Circuito_integrado)
- [11] Colaboradores de Wikipedia. (2022a, abril 22). Sistema embebido. Wikipedia, la enciclopedia libre. [https://es.wikipedia.org/wiki/Sistema\\_embebido](https://es.wikipedia.org/wiki/Sistema_embebido)
- [12] QuantumLeap - QuantumLeap. (s. f.). QuantumLeap. <https://quantumleap.readthedocs.io/en/latest/>
- [13] de Panfilis, G. (2022, 6 julio). FIWARE - Open APIs for Open Minds. FIWARE. <https://www.fiware.org/>
- [14] Alcarria, R. (s. f.). GitHub - gisai/LoRa-middleware: Middleware to connect LoRa Data packets from Chirpstack (HTTP integration) to Orion Context broker. GitHub. Recuperado 21 de junio de 2022, de <https://github.com/gisai/LoRa-middleware>

- [15] API Walkthrough (v2) - Fiware-Orion. (s. f.). FIWARE NGSI APIv2 Walkthrough. [https://fiware-orion.readthedocs.io/en/1.4.0/user/walkthrough\\_apiv2/index.html](https://fiware-orion.readthedocs.io/en/1.4.0/user/walkthrough_apiv2/index.html)
- [16] Colegio oficial de ingenieros de telecomunicación. (2021, mayo). *INTERNET DE LAS COSAS E INTERNET DE TODO*. <https://coitaoc.org/wp-content/uploads/2021/06/Estado-del-Arte-IoT.pdf>
- [17] Banafa, A. (2018, 1 agosto). The Internet of Everything (IoE). OpenMind. <https://www.bbvaopenmind.com/en/technology/digital-world/the-internet-of-everything-ioe/>
- [18] Johnson, J. T. (2022, 15 marzo). 6 IoT architecture layers and components explained. IoT Agenda. <https://www.techtarget.com/iotagenda/tip/A-comprehensive-view-of-the-4-IoT-architecture-layers>
- [19] Sarangam, A. (2022, 23 junio). 7 IOT Layers That You Should Know in 2021. Jigsaw Academy. <https://www.jigsawacademy.com/4-layers-of-the-internet-of-things/>
- [20] Beningo, J. (2020, 21 enero). Cómo seleccionar y usar el módulo ESP32 con Wi-Fi/Bluetooth adecuado para una aplicación de IoT industrial. Digi-Key Electronics. <https://www.digikey.es/es/articles/how-to-select-and-use-the-right-esp32-wi-fi-bluetooth-module>
- [21] colaboradores de Wikipedia. (2020, 27 agosto). ESP32. Wikipedia, la enciclopedia libre. <https://es.wikipedia.org/wiki/ESP32>
- [22] colaboradores de Wikipedia. (2022a, abril 22). *Middleware*. Wikipedia, la enciclopedia libre. <https://es.wikipedia.org/wiki/Middleware>
- [23] Orion Context Broker (OCB) - Aprende FIWARE en Español. (s. f.). FIWARE. [https://fiware-training.readthedocs.io/es\\_MX/latest/ecosistemaFIWARE/ocb/](https://fiware-training.readthedocs.io/es_MX/latest/ecosistemaFIWARE/ocb/)
- [24] The ThingsBoard Authors. (s. f.). What is ThingsBoard? ThingsBoard. <https://thingsboard.io/docs/getting-started-guides/what-is-thingsboard/>